GAMING CASTLE

Online Gaming Centre Management System

Final Design Report

By

Kirisigan	SE/2021/007
Nirojan	SE/2021/014
Thivya	SE/2021/022
Jashvanth	SE/2021/055

A report submitted in partial fulfilment of the requirements for the degree of Bachelor of Science Honours in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Declaration

We hereby certify that this project and all the artefacts associated with it are our own work and have not been submitted before, nor are currently being submitted for any other degree program.

Full name of student 1:	Kirisigan Kannathasan
Student No:	SE/2021/007
Signature:	
Full name of student 2:	Nirojan Kunesan
Student No:	SE/2021/014
Signature:	
Full name of student 3:	Thivya Mahendran
Student No:	SE/2021/022
Signature:	

Full name of student 4: Jashvanth Balakirushnan

Student No: SE/2021/055

Signature: 

Name of supervisor: _____

Signature of supervisor: _____

Date: _____

Acknowledgements

We would like to express our sincere appreciation to the Software Engineering Teaching Unit of the Faculty of Science, University of Kelaniya, for providing the guidelines and structure necessary to prepare this design document.

We extend our gratitude to our supervisor for the continuous guidance, constructive feedback, and valuable suggestions provided throughout the preparation of this report. Their support greatly contributed to improving the quality and clarity of the document.

We also acknowledge the cooperation of the client and stakeholders who provided the necessary information, requirements, and insights that helped in shaping the system design and documentation.

Furthermore, we are thankful for all academic resources, reference materials, and tools that supported us in understanding and applying system design concepts effectively.

This document represents our effort in compiling and presenting the system design in accordance with the given academic standards.

Abstract

The Gaming Castle Online Gaming Centre Management System is a web-based application designed to digitalise and streamline the operations of a gaming hub in Sri Lanka. This report presents the complete system design and implementation approach, developed based on the requirements defined in the Software Requirements Specification (SRS).

The system supports core functionalities including slot booking, tournament registration, payment processing, loyalty management, and administrative control. It is implemented using a microservices architecture with services such as User, Booking, Payment, Tournament, and Loyalty, ensuring scalability and maintainability. The backend is developed using Spring Boot, while the frontend uses Angular, and data is managed through a PostgreSQL database.

Security is enforced using JWT authentication, Role-Based Access Control (RBAC), and encryption techniques. The system is deployed using Docker containerisation and AWS cloud services, supported by a CI/CD pipeline.

This report includes architectural design, database design, class and sequence diagrams, UI/UX design, security mechanisms, and deployment strategies, providing a complete technical blueprint for the system.

Keywords: Gaming Management System, Microservices Architecture, Slot Booking, JWT Authentication, Cloud Deployment, Web Application.

Table of Contents

List of Tables

List of Figures

Abbreviations

Abbreviation	Definition
SRS	Software Requirements Specification
SDS	System Design Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
Admin	System Administrator
PS5	PlayStation 5
UI	User Interface

API	Application Programming Interface
IEEE	Institute of Electrical and Electronics Engineers
ISO/IEC	International Organisation for Standardisation / International Electrotechnical Commission
MVP	Minimum Viable Product
UAT	User Acceptance Testing
MVC	Model-View-Controller
REST	Representational State Transfer
TLS	Transport Layer Security
HTTPS	Hypertext Transfer Protocol Secure
WCAG	Web Content Accessibility Guidelines
OWASP	Open Worldwide Application Security Project
CI/CD	Continuous Integration / Continuous Deployment

JWT	JSON Web Token
RBAC	Role-Based Access Control

Table 1: Definitions, Acronyms and Abbreviations

Chapter 1 Introduction

This chapter introduces the Gaming Castle Online Gaming Centre Management System project. It describes the client, the problem being addressed, the proposed solution, the key stakeholders, and includes the client agreement letter. This foundational context is necessary for understanding the subsequent chapters of the report.

1.1 The Client Details

The client for this project is Gaming Castle, a modern gaming hub located in Nelliyadi, Sri Lanka. The centre is operated by its owner, Lathurshigan, and currently manages two PlayStation 5 (PS5) consoles offered on a pay-as-you-go basis. Gaming Castle serves local gaming enthusiasts by providing access to premium console gaming experiences and competitive tournament events.

Field	Details
Organisation Name	Gaming Castle
Location	Nelliyadi, Sri Lanka
Client Representative	Lathurshigan
Designation	Owner
Contact	Lathurshigan05@gmail.com +94 76 123 7419

1.2 Problem Statement

Gaming Castle currently relies entirely on manual processes for its core operations. Slot bookings are managed via phone calls and WhatsApp messages, payments are collected and tracked manually, tournament registrations are handled informally, and no structured loyalty programme is in place. These manual approaches result in operational inefficiencies including double bookings, payment tracking errors, a lack of customer engagement mechanisms, and limited business visibility for the owner.

The absence of a centralised digital system means that customer satisfaction is adversely affected, revenue opportunities are missed, and the business is unable to scale effectively. A clear need exists for an integrated online management platform to address these challenges.

1.3 Proposed Solution Overview

The proposed solution is a web-based Online Gaming Centre Management System developed specifically for Gaming Castle. The system provides an integrated platform for managing all core business operations digitally. The key capabilities are:

- Online slot booking for registered customers with real-time PS5 console availability display
- Tournament management including creation, registration, bracket management, and results publication
- Online and cash payment processing with digital receipt generation
- A customer loyalty programme that awards and enables redemption of points for discounts
- An administrative dashboard providing real-time operational insights, reporting, and user management

The system is developed as a web application using Angular.js (frontend), Spring Boot (backend), PostgreSQL (database), and a microservices architecture deployed on cloud infrastructure.

1.4 Stakeholders

Stakeholder	Role	Interest in System

Gaming Castle Owner	Client / Primary Stakeholder	Oversees business operations, monitors revenue, ensures business growth.
Administrator (Staff)	System User – Admin Role	Manages bookings, tournaments, payments, and reports.
Registered Customers	System User – Customer Role	Books slots, joins tournaments, earns loyalty rewards.
Walk-in Customers	Indirect User	Uses the system indirectly through administrator-assisted booking.
Development Team	System Developers	Responsible for designing, developing, testing, and deploying the system.
Project Supervisor	Academic Stakeholder	Provides academic oversight and evaluation.
University of Kelaniya	Academic Institution	Sets degree requirements and quality standards.

Table 2: Stakeholders

1.5 Client Agreement Letter

Date: 13/03/2026

To Whom It May Concern,

This letter confirms that **Lathurhsigan**, representing **Gaming Castle (Pvt) Ltd,** have agreed to collaborate with **Kirisigan-SE/2021/007, Nirojan-SE/2021/014, Thivya-SE/2021/022, Jashvanth-SE/2021/055** of **Faculty of Science, University of Kelaniya, Sri Lanka** for the purpose of developing a software-based system as part of their academic System Design Project.

1.5.1 Project Title

Gaming Castle

1.5.2. Background and Problem Statement

We have identified the need for a system to address the following problem:

- Manual booking system
- No tournament management systems
- No web portal for customers
- No revenue tracking
- No loyalty reward to the users

1.5.3. Purpose of the Proposed System

The proposed system is expected to:

- Web portal for booking the consoles
- Online tournament management through the website
- Admin panel to manage revenue and users details and tournaments

1.5.4. Scope of Collaboration

We agree to:

- Provide necessary information related to current processes
- Participate in requirement discussions and clarification meetings
- Review and validate the Software Requirement Specification (SRS)
- Provide feedback on the proposed design and prototype

The student developers agree to:

- Maintain confidentiality of all proprietary or sensitive information disclosed by the client
- Use client-provided information solely for the purposes of this academic project

- Provide regular progress updates and notify the client of any material changes to the project scope

1.5.5. Nature of the Project

This project is undertaken solely for academic purposes. We understand and agree that:

- **To the maximum extent permitted by applicable law, the university of Kelaniya and the student developers expressly disclaim all warranties, whether express, implied, statutory or otherwise, including any implied warranties of merchantability, fitness for a particular purpose, or non-infringement. The university and student developers shall have no liability whatsoever for any direct, indirect, incidental, special, consequential or punitive damages, data loss, or financial impact arising from or related to the use or inability to use this software.**

1.5.6. Confidentiality

- **"Confidential Information"** means any non-public data, documents, trade secrets, or business information disclosed by the client in connection with this project. The student developers and the University shall: (a) use Confidential Information solely for the purposes of this academic project; (b) not disclose it to any third party without the client's prior written consent; and (c) apply reasonable measures to protect it from unauthorized access. These obligations shall survive for a period of two (2) years following the completion or termination of the project. Confidential Information does not include information that is or becomes publicly known through no breach of this agreement, or that was already in the receiving party's possession prior to disclosure. Upon written request, all Confidential Information shall be returned or securely destroyed.

1.5.7. Intellectual Property

- All intellectual property rights in the software, code, designs, documentation, and other deliverables created by the student developers as part of this project ("Project IP") shall vest in and remain the property of the student developers and/or the University of Kelaniya. The client is granted a non-exclusive, non-transferable, royalty-free license to use the

Project IP solely for internal evaluation and testing purposes. The client shall not reproduce, distribute, sublicense, modify, or commercially exploit the Project IP without the prior written consent of the University. The client acknowledges that any data, logos, trademarks, or pre-existing materials provided by the client remain the client's exclusive property.

1.5.8. Term and Termination

- This agreement shall commence on the date of signing and shall remain in effect until the final submission of the project deliverables to the Faculty of Science, University of Kelaniya, or upon written notice of termination by either party. Either party may terminate this agreement by providing fourteen (14) days' written notice. Upon termination, the student developers shall return or destroy any Confidential Information in their possession, and the licence granted under Section 7 shall cease immediately.

I/We hereby confirm that the above project request is genuine and that we require the proposed solution as described.

Client Name: Lathurhsigan

Designation: Owner

Organisation: Gaming Castle

Contact Information: 0761237419

Signature: 

Official Seal: _____

Date of Signing: 15/03/2026

Chapter 2 Prototype Development and Client Validation

This chapter presents the high-fidelity prototype developed for the Gaming Castle system and documents the client validation activities conducted. It covers the UI/UX prototype with navigation flow, evidence of client involvement, ethical considerations, and a reflection on stakeholder communication throughout the project. Chapter 3 established the system design; this chapter confirms that the design has been validated with the client before proceeding to development.

2.1 UI/UX Design and Prototype

A high-fidelity interactive prototype was developed using Figma to demonstrate the key user interfaces and navigation flows of the Gaming Castle system. The prototype covers customer-facing interface.

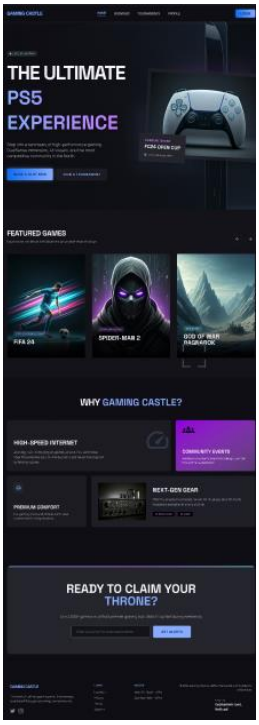


Figure 1: UI/UX Design-Home page

Figure 2: UI/UX Design-Login page

Figure 3: UI/UX Design-Register Page

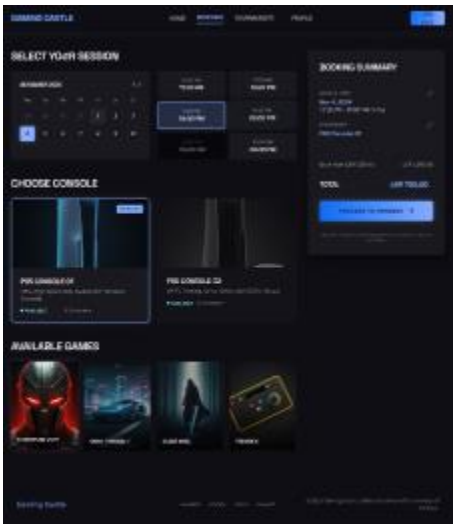


Figure 4: UI/UX Design-Booking page

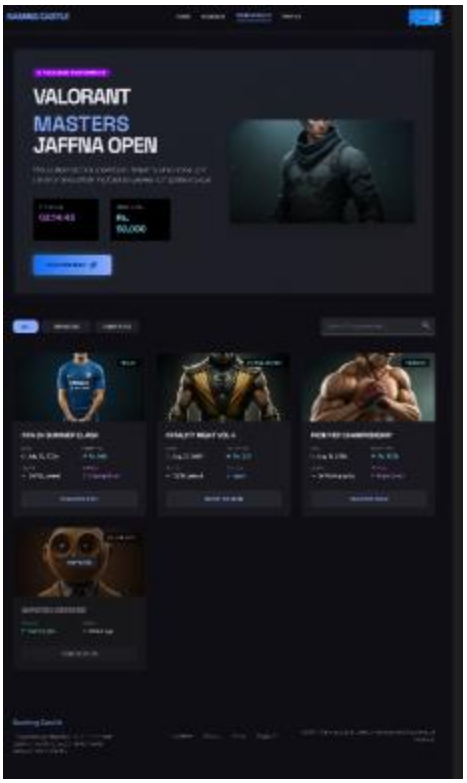


Figure 5: UI/UX Design-Tournament page

Action	**Navigation**
User Registration	Home page → Login → Create new Account → Enter details → Register
User Login	Home page → Login → Enter credentials → Login
Book slot	Home page → Book a slot → Select slot → Proceed to payment OR Booking page → Book a slot → Select slot → Proceed to payment
Register to a tournament	Home page → Join a tournament → Select the tournament → Payment page OR Tournament page → Join a tournament → Select the tournament → Payment page

Table 3: UI and UX

The prototype link is accessible here.

2.2 Client Involvement

2.2.1 Client Demonstration Session

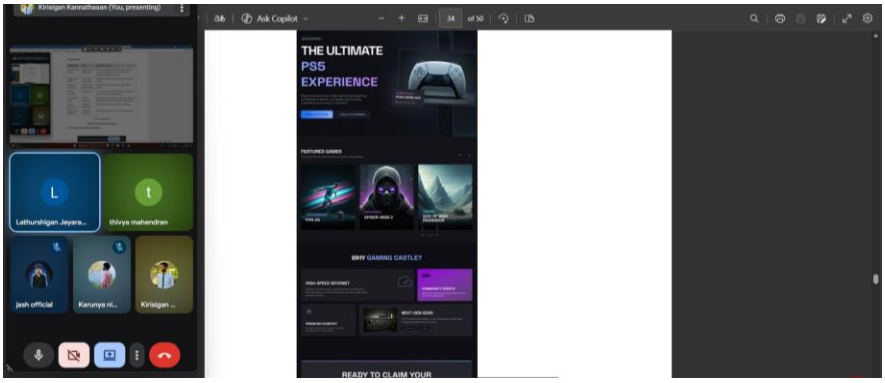


Figure 6: Client Demonstration Session 01

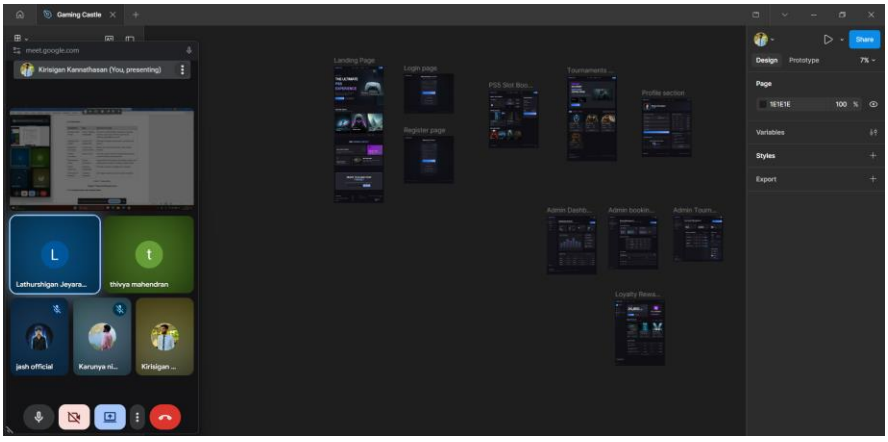


Figure 7: Client Demonstration Session 02

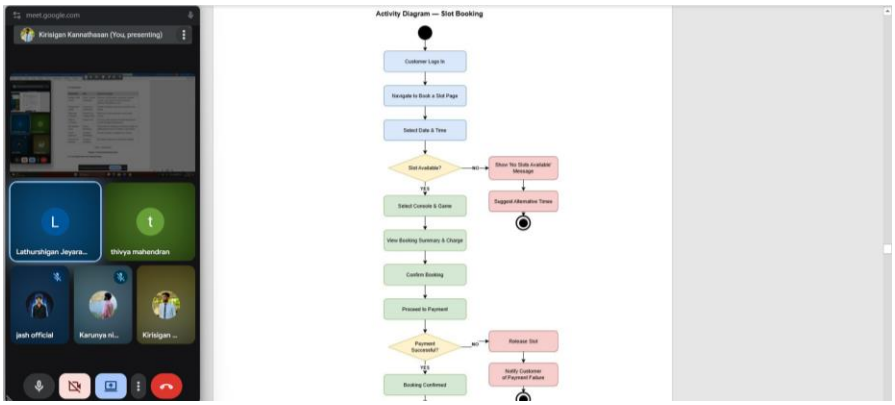


Figure 8: Client Demonstration Session 03

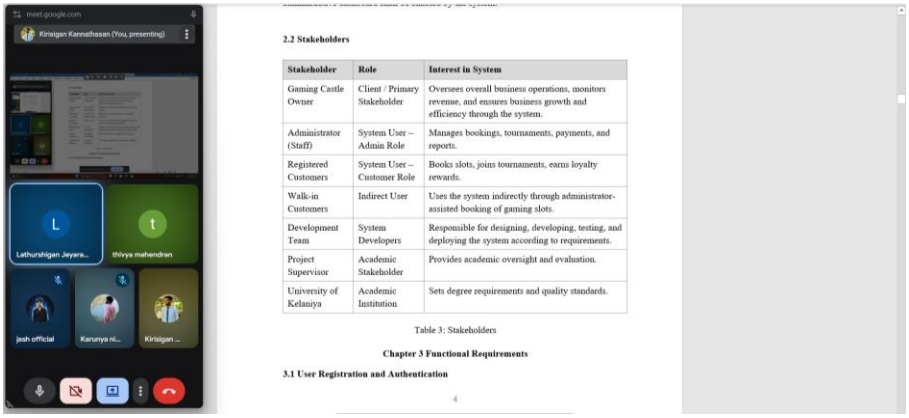
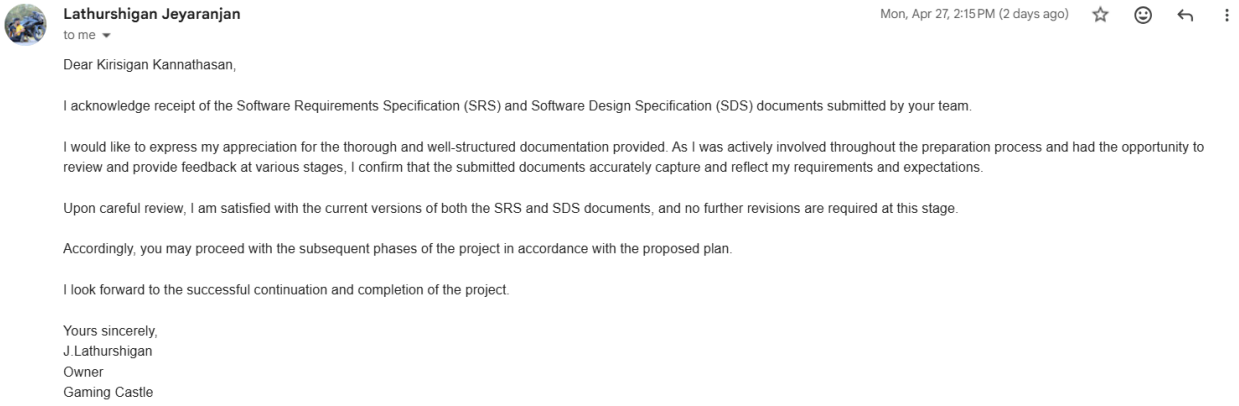


Figure 9: Client Demonstration Session 04

2.2.2 Client Feedback Report



Lathurshigan Jeyaranjan Mon, Apr 27, 2:15 PM (2 days ago)

to me

Dear Kirisigan Kannathasan,

I acknowledge receipt of the Software Requirements Specification (SRS) and Software Design Specification (SDS) documents submitted by your team.

I would like to express my appreciation for the thorough and well-structured documentation provided. As I was actively involved throughout the preparation process and had the opportunity to review and provide feedback at various stages, I confirm that the submitted documents accurately capture and reflect my requirements and expectations.

Upon careful review, I am satisfied with the current versions of both the SRS and SDS documents, and no further revisions are required at this stage.

Accordingly, you may proceed with the subsequent phases of the project in accordance with the proposed plan.

I look forward to the successful continuation and completion of the project.

Yours sincerely,
J.Lathurshigan
Owner
Gaming Castle

Figure 10: Client Feedback Report

2.3 Ethical Considerations

Data Privacy and Security

Customer personal data collected by the system is handled in compliance with applicable data protection principles. Passwords are stored exclusively as bcrypt hashes and all data transmissions are encrypted using TLS 1.2+. Client data shared during the project is treated in strict confidence per the Client Agreement Letter.

Informed Consent

The client entered into a formal agreement with full understanding of the project scope, their rights regarding intellectual property, and the academic nature of the project. All collaboration was conducted with the client's explicit consent and approval at each milestone.

Fairness and Accessibility

The system adheres to WCAG 2.1 Level AA guidelines to ensure accessibility for all users, including those with disabilities. The loyalty program and tournament features are designed to be transparent and fair to all registered customers.

Academic Integrity

All work presented in this report is the original work of the development team. All external sources and standards are properly cited in accordance with academic conventions.

Responsible Use of Technology

The system is developed solely for the academic purpose of managing a legitimate gaming business. No harmful, deceptive, or discriminatory features are incorporated. Only data necessary for the system's stated functions is collected.

2.4 Reflection on Stakeholder Communication

Review System

During requirement discussions, the client suggested adding a review system to allow customers to provide feedback on their gaming experience. Initially, this requirement was not identified. After discussing its benefits, such as improving service quality and customer engagement, the feature was incorporated into the system design. This

highlighted the importance of actively listening to stakeholder suggestions beyond initial requirements.

Messaging System

The client requested a messaging system to communicate with customers regarding bookings and updates. This requirement emerged later in the communication process. It required additional clarification to understand whether it should be real-time or notification-based. Finally, a simplified messaging/notification approach was included in the design. This emphasized the need for clear requirement clarification before implementation.

Booking Improvements

The client provided feedback that the booking process should be more user-friendly and faster. Based on this, the design was revised to reduce steps and improve the interface flow. This demonstrated how iterative feedback helps refine system usability.

2.5 Formal Sign-Off from the Client for the Development

The client has reviewed the final prototype and system design and has provided formal sign-off to proceed with full system development.

Client Name: Lathurshigan

Designation: Owner

Organization: Gaming Castle

Date: 30/04/2026

Signature: 

Official Seal: _____

Chapter 3 Conclusion

This chapter draws together the principal findings, design decisions, and learning outcomes for the Gaming Castle Online Gaming Centre Management System. It is important to note that this project represents the design phase of software development only; the system has not yet been implemented. The preceding chapters established the client context and problem statement (Chapter 1), captured and validated all software requirements (Chapter 2), produced a comprehensive system design specification (Chapter 3), and documented prototype development together with client validation activities (Chapter 4). This chapter evaluates how well the design objectives were met, reflects on the quality attributes addressed within the design, identifies limitations inherent to a design-only deliverable, summarizes the project timeline, and outlines the scope of future work to be carried out during the implementation phase.

3.1 Degree of Objectives Met

The objectives of this project were confined to the design phase: to produce a validated, well-structured, and client-approved set of design artefacts that would serve as a reliable blueprint for future development. The table below maps each design objective to the artefacts produced and assesses the extent to which each was achieved within the scope of this course.

Design Objective	Artefacts Produced	Degree Achieved
Define and validate software requirements with the client	Functional Requirements (FR-01 to FR-31), NFRs, Use Case Diagrams, Activity Diagrams, Risk Register, Client Sign-off	Fully achieved - all requirements were reviewed and formally approved by the client.
Design a scalable system architecture	Microservices architectural diagram, justification document, cloud-native deployment plan (AWS, Docker)	Fully achieved - a microservices architecture with independent User, Booking, Payment, Tournament, and Loyalty services was designed and justified.
Design the data model	ER Diagram, normalization explanation up to 3NF, PostgreSQL schema overview	Fully achieved - all core entities and relationships were modelled.
Produce detailed interaction designs	Class Diagram, Sequence Diagrams for all major use cases (slot booking, payment, tournament registration, authentication)	Fully achieved - all major use cases have corresponding sequence diagrams.
Design the user interface and experience	Figma high-fidelity interactive prototype with navigation flow	Fully achieved - prototype covers all customer-facing flows and was demonstrated to and approved by the client.
Address security requirements in the design	Security Design section covering JWT authentication, RBAC, bcrypt password hashing, TLS 1.2+, input validation, AES-256 encryption	Fully achieved - security design aligns with OWASP Top 10 and the NFRs specified in the SRS.

Define a deployment strategy	Hosting plan, environment specification (Development / Staging / Production), network diagram, CI/CD pipeline overview	Fully achieved - deployment architecture on AWS with Docker containerization is documented.

Table 4: Degree of Objectives Met

3.2 Usability, Accessibility, Reliability, and Friendliness

Although not yet implemented, the system design focuses on key quality attributes to ensure a high-quality user experience.

Usability:
The system is designed for ease of use, allowing first-time users to complete a booking within five minutes. It includes a clear step-by-step flow, simple controls, real-time availability, and user-friendly error messages. Admin dashboards are simplified to reduce navigation effort.

Accessibility:
The design follows WCAG 2.1 Level AA standards, ensuring compatibility across devices (mobile, tablet, desktop). It includes proper structure, good color contrast, and keyboard navigation to support users with disabilities.

Reliability:
A microservices architecture ensures system stability by isolating failures. AWS deployment, Docker containers, and health checks aim to achieve 99% uptime and quick recovery. These will be validated after implementation.

User-Friendliness:
The system provides real-time feedback such as loyalty points and booking confirmations via email/SMS. The design prioritizes simplicity, clarity, and a smooth user experience.

3.3 Limitations and Drawbacks

As a design-only project, several limitations exist:

Unvalidated design assumptions:
Key decisions (database performance, microservices communication, CI/CD effectiveness) are based on best practices but not yet tested.

Limited prototype scope:
The prototype focuses mainly on customer features; admin functionalities were not fully interactive or validated.

Incomplete cash payment workflow:
Details like reconciliation and handling discrepancies are not fully defined.

Limited scalability analysis:
The system is designed for two PS5 consoles; scaling to different devices is not fully explored.

Limited stakeholder input:
Requirements were gathered mainly from one client, without direct end-user research.

No detailed testing strategy:
While CI/CD and security checks are mentioned, a full testing plan is not yet defined.

3.4 Timeline

The project was carried out across four sequential stages as prescribed. All stages relate exclusively to design and documentation activities.

Stage	Deliverable	Key Activities	Outcome
1	Client Agreement & Problem Validation	Identified the client, conducted initial meetings to understand business needs and challenges, confirmed project feasibility, and completed the Project Concept Poster and signed Client Agreement Letter.	Client agreement secured; problem statement validated.

2	Software Requirement Specification (SRS)	Conducted requirement elicitation, analyzed the existing system, documented functional and non-functional requirements, created use case and activity diagrams, performed feasibility and risk analysis, and obtained client approval	SRS completed and formally approved by the client.
3	System Design Specification (SDS)	Designed and justified the microservices architecture, created ER, class, and sequence diagrams, and developed UI/UX wireframes along with security and AWS Docker-based deployment design.	SDS completed covering all required design artefacts.
4	Final Design Report & Prototype Validation	Developed a high-fidelity Figma prototype for customer flows, conducted a client demo and collected feedback, documented ethical and communication reflections, and finalized the design report with client approval to proceed.	Prototype validated; final report submitted; client sign-off obtained.

Table 5: Timelines

Client sign-off gates between the SRS and SDS stages ensured that each phase was validated before design work proceeded. This iterative validation approach helped align the design with the client's expectations and minimized the risk of significant rework during the development phase.

3.5 Future Modifications, Improvements, and Extensions

The following extensions and improvements are recommended for future development phases:

- Mobile application development (iOS/Android) to complement the web application.
- Integration with the PlayStation Network for automated game catalogue updates.
- Advanced analytics dashboard with predictive booking patterns for the owner.

- Expansion to support additional gaming consoles and game genres.
- Machine learning-based tournament bracket optimization.
- Multi-language support (Tamil and Sinhala) for the customer-facing interface.
- Online leaderboard and player statistics tracking.

References

[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[2] ISO/IEC, "Systems and software engineering – SQuaRE – System and software quality models," ISO/IEC 25010:2011, 2011.

[3] OWASP Foundation, "OWASP Top Ten," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. (accessed Mar. 11, 2026).

[4] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. (accessed Mar. 11, 2026).

[5] [Add additional references used in your design work here, following the same IEEE format.]

Appendix B - Software Requirement Specification (SRS)

This chapter presents the Software Requirement Specification (SRS) for the Gaming Castle Online Gaming Centre Management System. The SRS was developed in accordance with IEEE Std 830-1998 and quality requirements are aligned with ISO/IEC 25010. It was reviewed and formally approved by the client representative (Lathurshigan, Owner, Gaming Castle) on 16 April 2026.

B.1 Scope and Purpose

B.1.1 Purpose

This Software Requirements Specification (SRS) document is produced to define the functional and non-functional requirements for the Gaming Castle Online Gaming Centre Management System. A clear and comprehensive description of the system to be developed is provided by this document, and it is intended to serve as a contractual agreement between the development team and the client.

The intended audience of this document is identified as the project development team, academic evaluators, and the client (Gaming Castle management). The design, development, testing, and deployment phases of the system are guided by this document.

B.1.2 Scope

The system to be developed is named Gaming Castle, and it is designed as an Online Gaming Centre Management Web Application to support the operations of the Gaming Castle gaming hub located in Nelliyadi, Sri Lanka.

An integrated platform for managing slot bookings, tournament registrations, payment processing, customer loyalty programs, and administrative operations shall be provided by the system. PlayStation 5 (PS5) gaming slots shall be allowed to be booked online by registered customers, and participation in tournaments, as well as access to a loyalty reward system, shall be enabled.

Two primary user roles, namely Customer and Administrator, shall be supported by the system. Registration, login, gaming slot reservations, tournament participation, and the viewing of booking and reward history shall be enabled for customers. Booking management, tournament organization and supervision, activity monitoring, and report generation for decision-making shall be carried out by administrators.

The system shall be developed as a web-based application, and accessibility through modern web browsers shall be ensured for both customers and administrators.

The development of games, game streaming services, integration with external gaming platforms such as the PlayStation Network, and hardware-level control of gaming consoles shall not be included in the system.

B.1.3 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition

SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
Admin	System Administrator – Gaming Castle staff managing the platform
User / Customer	Registered individual who books slots or participates in tournaments
Slot	A reserved time block for using a PS5 console at Gaming Castle
Tournament	A competitive gaming event organized and managed through the system
Loyalty Program	A rewards scheme offering discounts to repeat customers
PS5	PlayStation 5
UI	User Interface
API	Application Programming Interface
IEEE	Institute of Electrical and Electronics Engineers
ISO/IEC 25010	International standard for software product quality characteristics
MVP	Minimum Viable Product
UAT	User Acceptance Testing

Table B.1: Definitions, Acronyms and Abbreviations

B.1.4 Document Overview

This document is organized as follows:

- Chapter 1 – Introduction: Provides the purpose, scope, definitions, and overview of the document.
- Chapter 2 – Project Scope and Stakeholders: Describes project boundaries and identifies all key stakeholders.
- Chapter 3 – Functional Requirements: Lists all functional requirements grouped by system modules in tabular format.
- Chapter 4 – Non-Functional Requirements: Details measurable quality requirements categorized per ISO/IEC 25010.
- Chapter 5 – Business Rules, Constraints, and Assumptions: Documents operational rules and project limitations.
- Chapter 6 – Use Case Diagrams and Descriptions: Presents use cases for all system actors.
- Chapter 7 – Activity Diagrams: Illustrates key workflows within the system.
- Chapter 8 – External Interface Requirements: Specifies UI, hardware, software, and communication interfaces.
- Chapter 9 – Data Requirements: Defines the data entities and storage requirements.
- Chapter 10 – Alternative Solutions Considered: Discusses alternative approaches evaluated.
- Chapter 11 – Feasibility Study: Assesses technical, economic, operational, and schedule feasibility.
- Chapter 12 – Risk Analysis: Identifies and evaluates project risks with mitigation strategies.
- Chapter 13 – References: Lists all cited sources and standards.

B.2 Project Scope and Stakeholders

B.2.1 Project Scope

Gaming Castle is a modern gaming hub located in Nelliyadi, Sri Lanka, where two PS5 consoles are operated and a selection of popular games is offered on a pay-as-you-go basis. Manual processes are currently relied upon for booking, payments, tournament management, and customer offers, resulting in operational inefficiencies.

All key operations of Gaming Castle shall be digitized and streamlined by the proposed web-based system. Online slot booking, automated payment management, structured tournament registration, a loyalty discount program, and a comprehensive administrative dashboard shall be enabled by the system.

B.2.2 Stakeholders

Stakeholder	Role	Interest in System
Gaming Castle Owner	Client / Primary Stakeholder	Oversees overall business operations, monitors revenue, and ensures business growth and efficiency through the system.
Administrator (Staff)	System User – Admin Role	Manages bookings, tournaments, payments, and reports.
Registered Customers	System User – Customer Role	Books slots, joins tournaments, earns loyalty rewards.
Walk-in Customers	Indirect User	Uses the system indirectly through administrator-assisted booking of gaming slots.
Development Team	System Developers	Responsible for designing, developing, testing, and deploying the system according to requirements.
Project Supervisor	Academic Stakeholder	Provides academic oversight and evaluation.
University of Kelaniya	Academic Institution	Sets degree requirements and quality standards.

Table B.2: Stakeholders

B.3 Functional Requirements

B.3.1 User Registration and Authentication

ID	Description	Priority	Stakeholder	Status

FR-01	The system shall allow customers to register using their name, email address, password, and phone number.	High	Customer	Approved
FR-02	The system shall allow registered users to log in using their email and password.	High	Customer	Approved
FR-03	The system shall allow registered users to log in using their phone number and password.	High	Customer	Approved
FR-04	The system shall allow users to reset their passwords via email verification.	High	Customer	Approved
FR-05	The system shall allow users to reset their passwords via phone number verification.	High	Customer	Approved
FR-06	The system shall allow the administrator to manage user accounts (create, update, deactivate).	High	Admin	Approved

Table B.3: Functional Requirements – User Registration and Authentication

B.3.2 Slot Booking Management

ID	Description	Priority	Stakeholder	Status
FR-07	The system shall display available time slots for PS5 consoles in real-time.	High	Customer	Approved
FR-08	The system shall allow registered customers to book a gaming slot by selecting a date, time, and console.	High	Customer	Approved

FR-09	The system shall allow the administrator to book slots on behalf of walk-in customers.	High	Admin	Approved
FR-10	The system shall prevent double-booking of the same console slot.	High	Customer	Approved
FR-11	The system shall allow customers to cancel or reschedule a booked slot within a defined time window.	High	Customer	Approved
FR-12	The system shall send booking confirmation notifications to customers via email or SMS.	Medium	Customer	Approved

Table B.4: Functional Requirements – Slot Booking Management

B.3.3 Tournament Management

ID	Description	Priority	Stakeholder	Status
FR-13	The system shall allow the administrator to create and publish tournament events with details (game, date, entry fee, max participants).	High	Admin	Approved
FR-14	The system shall allow registered customers to view and register for upcoming tournaments.	High	Customer	Approved
FR-15	The system shall manage tournament brackets and participant lists.	Medium	Admin	Approved

FR-16	The system shall notify registered participants of tournament schedules and results.	Medium	User	Approved
FR-17	The system shall allow the administrator to update tournament status (upcoming, ongoing, completed).	Medium	Admin	Approved
FR-18	The system shall include an automated tournament management module that generates tournament brackets and manages match progression based on predefined logic or historical data.	Medium	Admin	Approved

Table B.5: Functional Requirements – Tournament Management

B.3.4 Payment Management

ID	Description	Priority	Stakeholder	Status
FR-19	The system shall calculate charges based on booked hours at the applicable hourly rate.	High	Customer	Approved
FR-20	The system shall support online payment processing for slot bookings and tournament entry fees.	High	Customer	Approved
FR-21	The system shall generate and display digital receipts upon successful payment.	High	Customer	Approved
FR-22	The system shall allow the administrator to record cash payments for walk-in customers.	High	Admin	Approved

FR-23	The system shall provide a payment history log accessible to customers and the administrator.	Medium	Admin / Customer	Approved

Table B.6: Functional Requirements – Payment Management

B.3.5 Loyalty Program

ID	Description	Priority	Stakeholder	Status
FR-24	The system shall award loyalty points to customers for each completed booking.	High	Customer	Approved
FR-25	The system shall allow customers to redeem loyalty points for discounts on future bookings.	High	Customer	Approved
FR-26	The system shall display a customer's current loyalty points balance on their profile.	Medium	Customer	Approved
FR-27	The system shall allow the administrator to configure loyalty point earning and redemption rules.	Medium	Admin	Approved

Table B.7: Functional Requirements – Loyalty Program

B.3.6 Administrative Dashboard

ID	Description	Priority	Stakeholder	Status

FR-28	The system shall provide an administrative dashboard displaying real-time booking status, revenue summary, and active tournaments.	High	Admin	Approved
FR-29	The system shall allow the administrator to generate reports on bookings, revenue, and tournament participation.	High	Admin	Approved
FR-30	The system shall allow the administrator to manage game titles available for booking.	Medium	Admin	Approved
FR-31	The system shall allow the administrator to view and manage all registered customer accounts.	Medium	Admin	Approved

Table B.8: Functional Requirements – Administrative Dashboard

B.4 Non-Functional Requirements

All non-functional requirements are categorized in accordance with ISO/IEC 25010 and include measurable acceptance criteria.

B.4.1 Performance

ID	Description	Acceptance Criterion
NFR-01	The system shall respond to all page load requests efficiently.	95% of pages shall load within 3 seconds under normal load conditions.
NFR-02	The system shall handle concurrent users without degradation.	The system shall support at least 100 concurrent users without a measurable performance drop.
NFR-03	Slot availability updates shall be reflected in near real-time.	Slot status updates shall be visible to all users within 5 seconds of a change.

Table B.9: Non-Functional Requirements – Performance

B.4.2 Reliability

ID	Description	Acceptance Criterion
NFR-04	The system shall maintain high availability.	The system shall achieve a minimum uptime of 99% per month, excluding planned maintenance windows.
NFR-05	The system shall recover from failures gracefully.	In the event of an unexpected failure, the system shall restore full system functionality within 30 minutes.

Table B.10: Non-Functional Requirements – Reliability

B.4.3 Usability

ID	Description	Acceptance Criterion
NFR-06	The system shall be easy to use for first-time users.	A first-time user shall be able to complete a slot booking within 5 minutes without external assistance.
NFR-07	The system shall support responsive design.	The system shall render correctly on desktop, tablet, and mobile screen sizes (minimum 320px width).
NFR-08	The system shall display error messages clearly.	All user-facing error messages shall be written in plain English and suggest a corrective action.

Table B.11: Non-Functional Requirements – Usability

B.4.4 Security

ID	Description	Acceptance Criterion

NFR-09	All data transmissions shall be encrypted.	The system shall enforce HTTPS (TLS 1.2 or higher) for all communications.
NFR-10	User passwords shall be stored securely.	Passwords shall be hashed using bcrypt or an equivalent algorithm; plain-text storage is prohibited.
NFR-11	The system shall enforce role-based access control.	Administrative functions shall be accessible only to authenticated admin accounts; customer accounts shall not access admin panels.
NFR-12	The system shall protect against common web vulnerabilities.	The system shall pass an OWASP Top 10 vulnerability assessment prior to production deployment.

Table B.12: Non-Functional Requirements – Security

B.4.5 Scalability

ID	Description	Acceptance Criterion
NFR-13	The system shall be designed to accommodate future growth.	The system architecture shall support horizontal scaling to handle a 3x increase in user load without a redesign.
NFR-14	The database shall handle increasing data volumes efficiently.	Query response times shall remain within 2 seconds under normal conditions as the database grows to 100,000 booking records.

Table B.13: Non-Functional Requirements – Scalability

B.4.6 Maintainability

ID	Description	Acceptance Criterion
NFR-15	The codebase shall be well-documented.	All modules, functions, and APIs shall include inline comments and a README; documentation coverage shall exceed 80%.

NFR-16	The system shall support modular architecture.	Each major feature (booking, payments, tournaments, loyalty) shall be implemented as an independently maintainable module.

Table B.14: Non-Functional Requirements – Maintainability

B.4.7 Portability

ID	Description	Acceptance Criterion
NFR-17	The system shall operate across major web browsers.	The system shall function correctly on the latest versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
NFR-18	The system shall be deployable on standard cloud infrastructure.	The system shall be deployable on platforms such as AWS, Google Cloud, or Azure with minimal configuration changes.

Table B.15: Non-Functional Requirements – Portability

B.5 Business Rules, Constraints, and Assumptions

B.5.1 Business Rules

- BR-01: A customer must be registered and logged in to make an online slot booking.
- BR-02: Each PS5 console may only be assigned to one customer per time slot.
- BR-03: Slot bookings shall be available in minimum increments of one hour.
- BR-04: Cancellations made more than two hours before the slot start time shall be eligible for a full refund or rescheduling.
- BR-05: Loyalty points shall be awarded only upon completion and confirmation of a paid booking.
- BR-06: Tournament registration shall close once the maximum participant limit is reached or the registration deadline passes.
- BR-07: Override capability shall be provided to the administrator to allow any booking to be cancelled or reassigned.
- BR-08: Walk-in customer bookings recorded by the admin shall default to cash payment.

B.5.2 Constraints

- The system must be developed as a web application accessible via standard web browsers.
- The initial deployment shall support 2 PS5 consoles; the system shall be scalable to accommodate additional consoles in the future.
- The project must be completed within the academic semester timeline.
- The development team consists of 4 members; resource allocation is constrained accordingly.
- The system shall be built using open-source or freely available frameworks to minimize licensing costs.
- Internet connectivity is required for the system to function; offline mode is not within the project scope.

B.5.3 Assumptions

- It is assumed that Gaming Castle management will provide timely feedback and sign-off at each project milestone.
- It is assumed that all customers have access to a device with internet connectivity and a modern web browser.
- It is assumed that the hourly rates and game titles will be provided by the client prior to system development.
- It is assumed that a payment gateway (e.g., PayHere or Stripe) will be available and accessible for integration.
- It is assumed that the Gaming Castle premises will have stable internet connectivity for admin operations.
- It is assumed that sufficient predefined rules or sample data will be available to configure or simulate the automated tournament management module.

B.6 Use Case Diagrams and Descriptions

B.6.1 System Actors

The Gaming Castle system involves two primary actors:

- Customer (Registered User): A person who accesses the system online to book slots, register for tournaments, manage their profile, and redeem loyalty rewards.
- Administrator (Admin): A member of Gaming Castle staff who manages bookings, tournaments, payments, customer accounts, and shop operations through the admin dashboard.

B.6.2 Use Case Summary

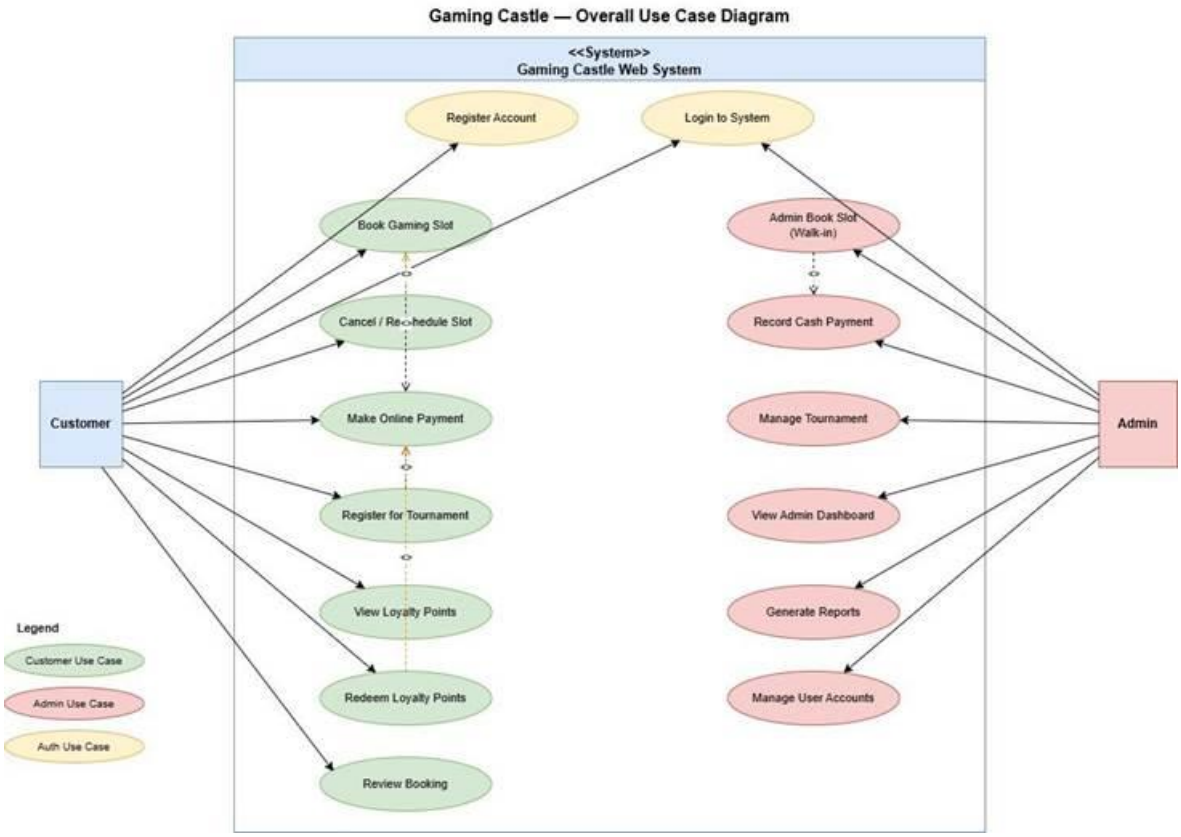


Figure B.1: Overall Use Case Diagram – Gaming Castle Web System

Use Case ID	Use Case Name	Actor(s)	Priority
UC-01	Register Account	Customer	High

UC-02	Login to the System	Customer / Admin	High
UC-03	Book Gaming Slot	Customer	High
UC-04	Cancel / Reschedule Slot	Customer	Medium
UC-05	Admin Book Slot for Walk-in Customer	Admin	High
UC-06	Make Online Payment	Customer	High
UC-07	Record Cash Payment	Admin	High
UC-08	Register for Tournament	Customer	High
UC-09	Manage Tournament	Admin	High
UC-10	View Loyalty Points	Customer	Medium
UC-11	Redeem Loyalty Points	Customer	Medium
UC-12	View Admin Dashboard	Admin	High
UC-13	Generate Reports	Admin	Medium
UC-14	Manage User Accounts	Admin	Medium

Table B.16: Use Case Summary

B.6.3 Use Case Descriptions

Use Case UC-01: Register Account

Attribute	Details
Use Case ID	UC-01
Use Case Name	Register Account

Actor(s)	Customer
Preconditions	The customer has not previously registered. The system is accessible.
Main Flow	1. The customer navigates to the Register page. 2. Required details are entered (name, email, phone number, password). 3. The input fields are validated by the system. 4. It is verified by the system that the email or phone number is not already registered. 5. The account is created and the details are stored by the system. 6. A success message is displayed and the user is redirected to the Login page.
Alternative Flow	3a. If any field is invalid – the error is highlighted and correction is prompted by the system. 4a. If the email or phone number is already registered – the following message is displayed: An account with this email already exists.
Postconditions	A new customer account is created and stored in the system. The customer may now log in.
Exception Flow	If the system is unavailable, a service error message is displayed and the customer is prompted to try again later.
Related Requirements	FR-01, FR-02, FR-03

Table B.17: Use Case Description – UC-01: Register Account

Use Case UC-02: Login to System

Attribute	**Details**
Use Case ID	UC-02
Use Case Name	Login to System
Actor(s)	Customer / Admin

Preconditions	The user has a registered account. The system is accessible.
Main Flow	1. The user navigates to the Login page. 2. An email address and password are entered by the user. 3. The credentials are validated by the system. 4. The user is authenticated and the role (Customer or Admin) is identified by the system. 5. The user is redirected to the appropriate dashboard (Customer home or Admin dashboard).
Alternative Flow	3a. If credentials are invalid – an error message is displayed and the user is prompted to re-enter. 3b. If the password has been forgotten – the user selects Forgot Password and is redirected to the password reset flow.
Postconditions	The user is authenticated and granted access to role-appropriate features.
Exception Flow	If the account is deactivated, the following message is displayed: Your account has been deactivated. Please contact support.
Related Requirements	FR-02, FR-03, FR-04, FR-05

Table B.18: Use Case Description – UC-02: Login to System

Use Case UC-03: Book Gaming Slot

Attribute	**Details**
Use Case ID	UC-03
Use Case Name	Book Gaming Slot
Actor(s)	Customer
Preconditions	The customer is registered and logged into the system.

Main Flow	1. The customer navigates to the Book a Slot page. 2. The preferred date and time are selected. 3. An available PS5 console and game are selected. 4. The booking summary with the applicable charge is displayed by the system. 5. The booking is confirmed and payment is proceeded with by the customer. 6. Payment is processed and the booking is confirmed by the system. 7. A confirmation notification is sent to the customer.
Alternative Flow	If no slots are available for the selected time, the customer is notified and alternative times are suggested by the system.
Postconditions	A confirmed booking record is created. The slot is marked as unavailable for other users.
Exception Flow	If payment fails, the slot is released and the customer is notified.
Related Requirements	FR-07, FR-08, FR-10, FR-12, FR-19, FR-20, FR-21

Table B.19: Use Case Description – UC-03: Book Gaming Slot

Use Case UC-04: Cancel / Reschedule Slot

Attribute	Details
Use Case ID	UC-04
Use Case Name	Cancel / Reschedule Slot
Actor(s)	Customer
Preconditions	The customer is registered and logged in. An existing confirmed booking is held by the customer.

Main Flow	1. The customer navigates to My Bookings. 2. A booking to cancel or reschedule is selected. 3. The system checks whether the cancellation/reschedule window is still open (more than 2 hours before slot start). 4. The cancellation is confirmed or a new date and time is selected by the customer. 5. The booking record is updated accordingly by the system. 6. The customer is notified of the change via email or SMS.
Alternative Flow	4a. If rescheduling – available slots are displayed for the customer to choose from. 4b. If cancelling – a refund is processed by the system if applicable.
Postconditions	The original slot is released and made available for other users. The booking record is updated or cancelled.
Exception Flow	If the cancellation window has passed (less than 2 hours before the slot), the customer is informed that the booking can no longer be modified.
Related Requirements	FR-11, FR-12

Table B.20: Use Case Description – UC-04: Cancel / Reschedule Slot

Use Case UC-05: Admin Book Slot for Walk-in Customer

Attribute	Details
Use Case ID	UC-05
Use Case Name	Admin Book Slot for Walk-in Customer
Actor(s)	Admin
Preconditions	The admin is authenticated and has admin-level access. At least one slot is available.

Main Flow	1. The admin navigates to the Admin Booking panel. 2. An available date, time, console, and game are selected. 3. The walk-in customer details are entered by the admin. 4. The booking is created and the slot is marked as occupied by the system. 5. The cash payment is recorded by the admin. 6. Revenue records are updated and a receipt is generated by the system.
Alternative Flow	2a. If no slots are available for the selected time – the admin is notified and alternative times are suggested.
Postconditions	A booking record is created for the walk-in customer. The slot is marked as occupied. Cash payment is recorded.
Exception Flow	If the system fails to save the booking, an error message is displayed and the admin is prompted to retry.
Related Requirements	FR-09, FR-22

Table B.21: Use Case Description – UC-05: Admin Book Slot for Walk-in Customer

Use Case UC-06: Make Online Payment

Attribute	**Details**
Use Case ID	UC-06
Use Case Name	Make Online Payment
Actor(s)	Customer
Preconditions	The customer is registered and logged in. A pending booking awaiting payment exists.

Main Flow	1. The booking summary with the total charge is displayed by the system. 2. An online payment method (card or gateway) is selected by the customer. 3. Payment details are entered and confirmed by the customer. 4. The payment is processed via the integrated payment gateway. 5. The payment is confirmed and the booking is finalized. 6. A digital receipt is generated and displayed. 7. A booking confirmation notification is sent to the customer.
Alternative Flow	3a. If the customer cancels the payment – the system returns to the booking summary and the slot remains reserved briefly before being released.
Postconditions	Payment is confirmed. The booking is finalized. Loyalty points are awarded to the customer.
Exception Flow	If the payment fails (e.g. insufficient funds, gateway error) – the customer is notified, the slot reservation is released, and a retry is prompted.
Related Requirements	FR-19, FR-20, FR-22, FR-23,FR-24

Table B.22: Use Case Description – UC-06: Make Online Payment

Use Case UC-07: Record Cash Payment

Attribute	**Details**
Use Case ID	UC-07
Use Case Name	Record Cash Payment
Actor(s)	Admin
Preconditions	The admin is authenticated. A walk-in booking has been created by the admin.

Main Flow	1. The admin navigates to the payment recording section. 2. The relevant booking is selected. 3. The cash amount received is entered by the admin. 4. The amount is validated by the system against the booking charge. 5. The booking is marked as paid by the system. 6. A receipt is generated and revenue records are updated.
Alternative Flow	3a. If the amount entered does not match the expected charge – the admin is alerted to confirm before proceeding.
Postconditions	The booking is marked as paid (cash). Revenue records are updated. A receipt is generated.
Exception Flow	If the system fails to record the payment, an error is displayed and the admin is prompted to retry.
Related Requirements	FR-21, FR-22

Table B.23: Use Case Description – UC-07: Record Cash Payment

Use Case UC-08: Register for Tournament

Attribute	Details
Use Case ID	UC-08
Use Case Name	Register for Tournament
Actor(s)	Customer
Preconditions	The customer is registered and logged in. A tournament with open registration exists.

Main Flow	1. The customer navigates to the Tournaments page. 2. Available upcoming tournaments are viewed. 3. A tournament is selected and its details are viewed (game, date, entry fee, max participants). 4. The customer selects Register and is directed to payment. 5. Payment of the entry fee is completed. 6. Registration is confirmed and the customer is added to the participant list. 7. A confirmation notification is sent to the customer.
Alternative Flow	5a. If payment fails – registration is not confirmed and the customer is notified.
Postconditions	The customer is registered for the tournament. Entry fee payment is recorded.
Exception Flow	If the tournament is full or the registration deadline has passed – the customer is informed that registration is closed.
Related Requirements	FR-14, FR-19

Table B.24: Use Case Description – UC-08: Register for Tournament

Use Case UC-09: Manage Tournament

Attribute	**Details**
Use Case ID	UC-09
Use Case Name	Manage Tournament
Actor(s)	Admin
Preconditions	The admin is authenticated and has admin-level access.

Main Flow	1. The admin navigates to the Tournament Management section. 2. A new tournament is created by entering details (name, game, date, entry fee, max participants). 3. The tournament is published and made visible to customers by the system. 4. Registrations are monitored in real-time by the admin. 5. Tournament status is updated as it progresses. 6. Tournament results are recorded and published.
Alternative Flow	The admin may cancel a tournament before it begins; all registered participants are notified automatically by the system.
Postconditions	Tournament records are created/updated in the system.
Exception Flow	If the maximum participant limit is reached, registration is automatically closed by the system.
Related Requirements	FR-13, FR-15, FR-16, FR-17

Table B.25: Use Case Description – UC-09: Manage Tournament

Use Case UC-10: View Loyalty Points

Attribute	**Details**
Use Case ID	UC-10
Use Case Name	View Loyalty Points
Actor(s)	Customer
Preconditions	The customer is registered and logged in.
Main Flow	1. The customer navigates to their Profile or Loyalty Rewards section. 2. The loyalty points balance is retrieved by the system. 3. The current points balance and a history of earned and redeemed points are displayed.

Alternative Flow	3a. If the customer has no points history – a balance of 0 is displayed along with a message encouraging the first booking.
Postconditions	The current loyalty points balance and transaction history are viewed by the customer.
Exception Flow	If the loyalty data cannot be loaded, an error is displayed and the customer is prompted to refresh the page.
Related Requirements	FR-25

Table B.26: Use Case Description – UC-10: View Loyalty Points

Use Case UC-11: Redeem Loyalty Points

Attribute	Details
Use Case ID	UC-11
Use Case Name	Redeem Loyalty Points
Actor(s)	Customer
Preconditions	The customer is registered and logged in. Sufficient loyalty points for redemption are available.
Main Flow	1. The customer navigates to the Slot Booking page and proceeds to payment. 2. The available loyalty points and equivalent discount are displayed by the system. 3. The customer opts to redeem points against the booking charge. 4. The discount is applied and the updated total is displayed . 5. Payment for the remaining balance is confirmed and completed. 6. The redeemed points are deducted from the customer's balance.
Alternative Flow	3a. If redemption is not opted for – the booking proceeds at full price with no points deducted.
Postconditions	Loyalty points are deducted. The booking charge is reduced by the redeemed amount.

Exception Flow	If the points balance drops below the minimum redemption threshold during checkout, the option is disabled and the customer is notified.
Related Requirements	FR-24, FR-25

Table B.27: Use Case Description – UC-11: Redeem Loyalty Points

Use Case UC-12: View Admin Dashboard

Attribute	Details
Use Case ID	UC-12
Use Case Name	View Admin Dashboard
Actor(s)	Admin
Preconditions	The admin is authenticated and has admin-level access.
Main Flow	1. The admin logs in and is redirected to the Admin Dashboard. 2. Real-time summary widgets are displayed: current active tournaments, today's bookings, revenue summary, and upcoming tournaments. 3. The admin navigates to detailed sections (Bookings, Payments, Tournaments, Users, Reports) from the dashboard.
Alternative Flow	2a. If there are no active sessions or bookings – empty state messages are displayed for the relevant widgets.
Postconditions	A real-time operational overview of the Gaming Castle system is viewed by the admin.

Exception Flow	If real-time data fails to load, the last cached data is displayed with a warning that data may not be current.
Related Requirements	FR-27

Table B.28: Use Case Description – UC-12: View Admin Dashboard

Use Case UC-13: Generate Reports

Attribute	Details
Use Case ID	UC-13
Use Case Name	Generate Reports
Actor(s)	Admin
Preconditions	The admin is authenticated and has admin-level access.
Main Flow	1. The admin navigates to the Reports section from the Admin Dashboard. 2. A report type is selected (Bookings, Revenue, Tournament Participation). 3. The date range and any filters are specified. 4. Relevant data is retrieved and processed by the system. 5. The report is displayed in a structured format. 6. The report may be exported (e.g., PDF or CSV) by the admin.
Alternative Flow	5a. If no data is available for the selected criteria – the admin is informed that no records match the filter.
Postconditions	A report is generated and optionally exported by the admin.
Exception Flow	If report generation fails due to a system error, the admin is notified and prompted to retry.
Related Requirements	FR-28

Table B.29: Use Case Description – UC-13: Generate Reports

Use Case UC-14: Manage User Accounts

Attribute	Details
Use Case ID	UC-14
Use Case Name	Manage User Accounts
Actor(s)	Admin
Preconditions	The admin is authenticated and has admin-level access.
Main Flow	1. The admin navigates to the User Management section. 2. A list of all registered customer accounts is displayed. 3. Users may be searched or filtered by name, email, or status. 4. A user is selected to view their profile, booking history, and loyalty points. 5. Account details may be updated or the account may be deactivated if required. 6. Changes are saved and reflected immediately by the system.
Alternative Flow	5a. If a user is deactivated – a notification is sent to the affected user by the system.
Postconditions	User account details are updated or the account is deactivated as required.
Exception Flow	If the system fails to save changes, an error is displayed and the admin is prompted to retry.
Related Requirements	FR-06, FR-30

Table B.30: Use Case Description – UC-14: Manage User Accounts

B.7 Activity Diagrams

This chapter presents eight key workflows within the Gaming Castle system. Each activity diagram illustrates the sequence of activities, decision points, and possible outcomes for a primary use case.

B.7.1 Slot Booking

This activity diagram describes the process by which a registered customer books a PS5 gaming slot through the Gaming Castle web system. The workflow covers slot availability checking, booking confirmation, and payment processing.

Step	Activity	Actor	Decision / Outcome
1	Customer Logs In	Customer	Precondition: Customer must be registered
2	Navigate to Book a Slot Page	Customer	–
3	Select Date & Time	Customer	–
4	Check Slot Availability	System	Decision: Slot Available? YES → Step 5 | NO → Show message, suggest alternatives → END
5	Select Console & Game	Customer	–
6	View Booking Summary & Charge	System	–
7	Confirm Booking	Customer	–
8	Proceed to Payment	System	–
9	Process Payment	System	Decision: Payment Successful? YES → Step 10 | NO → Release slot, notify customer → END
10	Booking Confirmed	System	–
11	Send Confirmation Notification	System	Postcondition: Slot marked unavailable

Table B.31: Activity Steps – Slot Booking

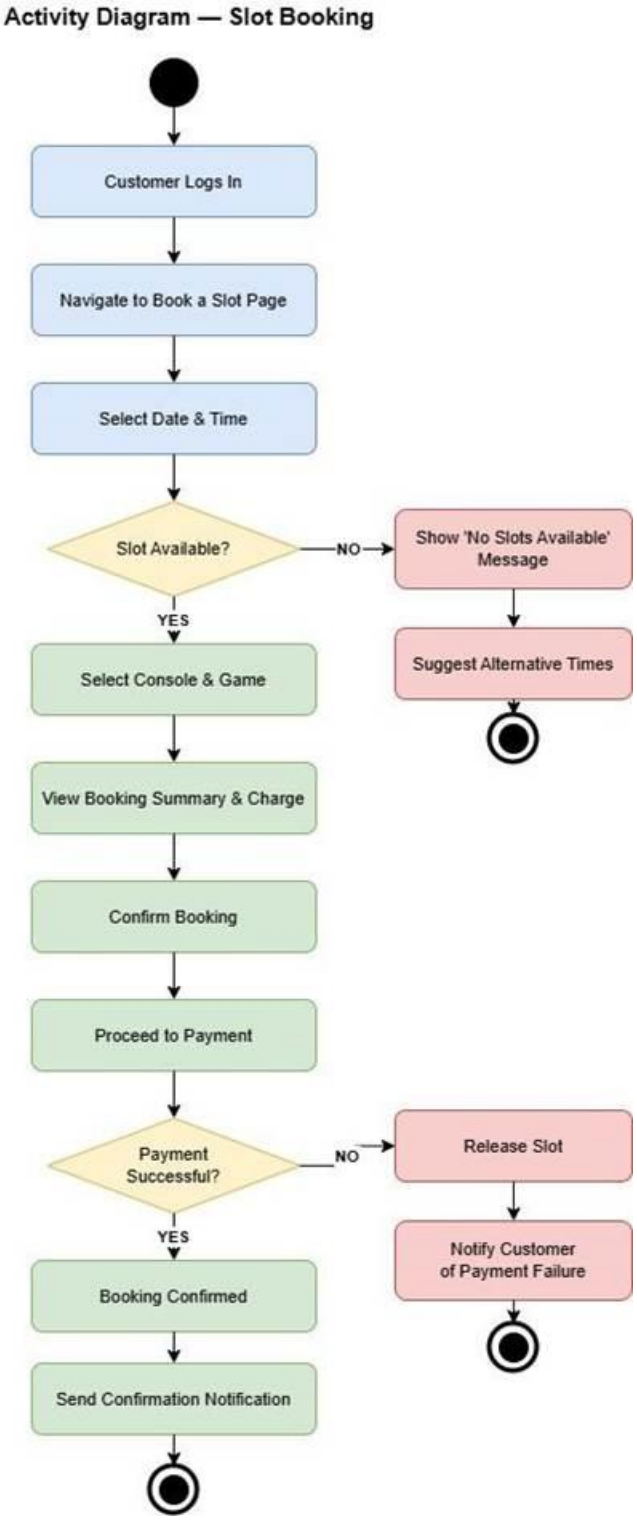


Figure B.2: Activity Diagram – Slot Booking

B.7.2 Tournament Registration

This activity diagram describes the process by which a registered customer views, selects, and registers for a tournament on the Gaming Castle platform, including entry fee payment validation.

Step	Activity	Actor	Decision / Outcome
1	Customer Logs In	Customer	Precondition: Customer must be registered
2	Navigate to Tournaments Page	Customer	–
3	View Available Tournaments	Customer	–
4	Select a Tournament	Customer	–
5	Check Registration Status	System	Decision: Registration Open? NO → Show Closed message → END
6	Check Participant Limit	System	Decision: Tournament Full? YES → Show Full message → END
7	Click Register	Customer	–
8	Proceed to Entry Fee Payment	System	–
9	Process Payment	System	Decision: Payment Successful? YES → Step 10 | NO → Notify customer → END
10	Registration Confirmed	System	–
11	Send Confirmation Notification	System	Postcondition: Participant count incremented

Table B.32: Activity Steps – Tournament Registration

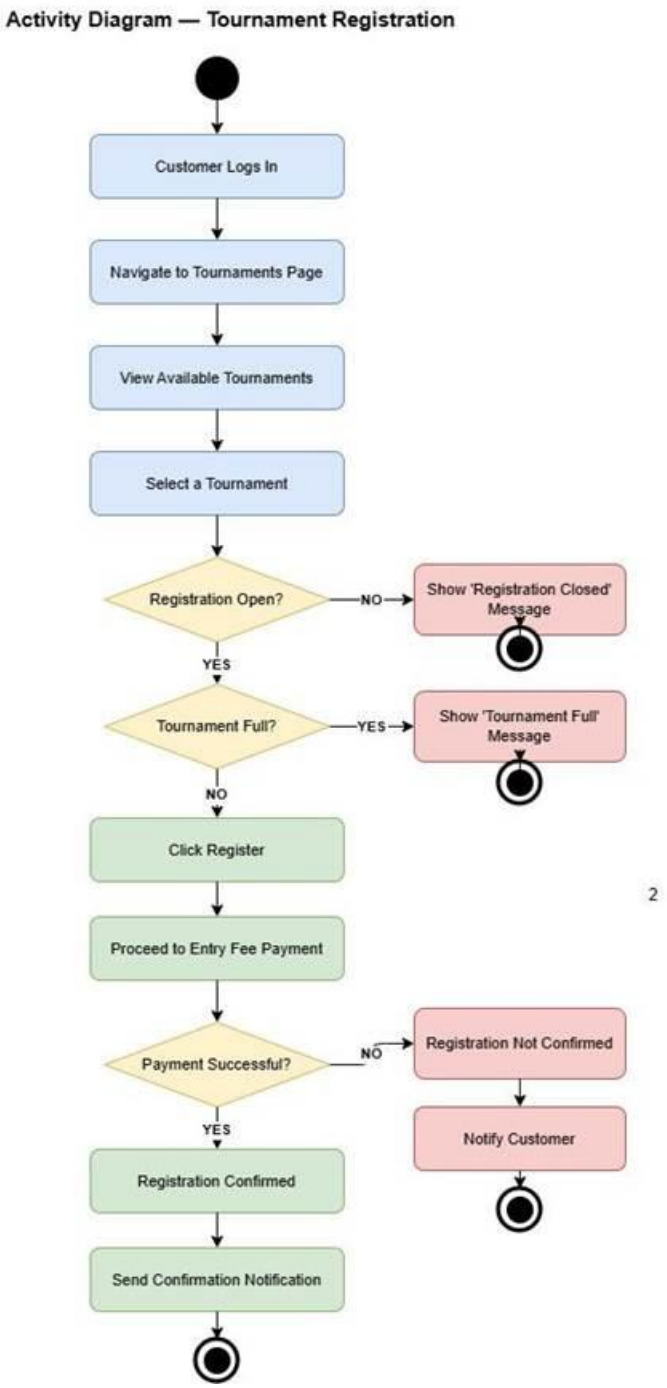


Figure B.3: Activity Diagram – Tournament Registration

B.7.3 Admin Walk-in Booking

This activity diagram describes the process by which an administrator books a PS5 gaming slot on behalf of a walk-in customer who does not use the online system, and records a cash payment.

Step	Activity	Actor	Decision / Outcome
1	Admin Logs In	Admin	Precondition: Admin must be authenticated
2	Navigate to Admin Booking Panel	Admin	–
3	View Available Slots	Admin	–
4	Check Slot Availability	System	Decision: Slot Available? NO → Notify admin → END
5	Select Slot, Console & Game	Admin	–
6	Enter Walk-in Customer Name	Admin	–
7	System Creates Booking	System	–
8	Mark Slot as Occupied	System	–
9	Admin Records Cash Payment	Admin	–
10	System Updates Revenue Records	System	–
11	Generate Receipt	System	Postcondition: Revenue records updated

Table B.33: Activity Steps – Admin Walk-in Booking

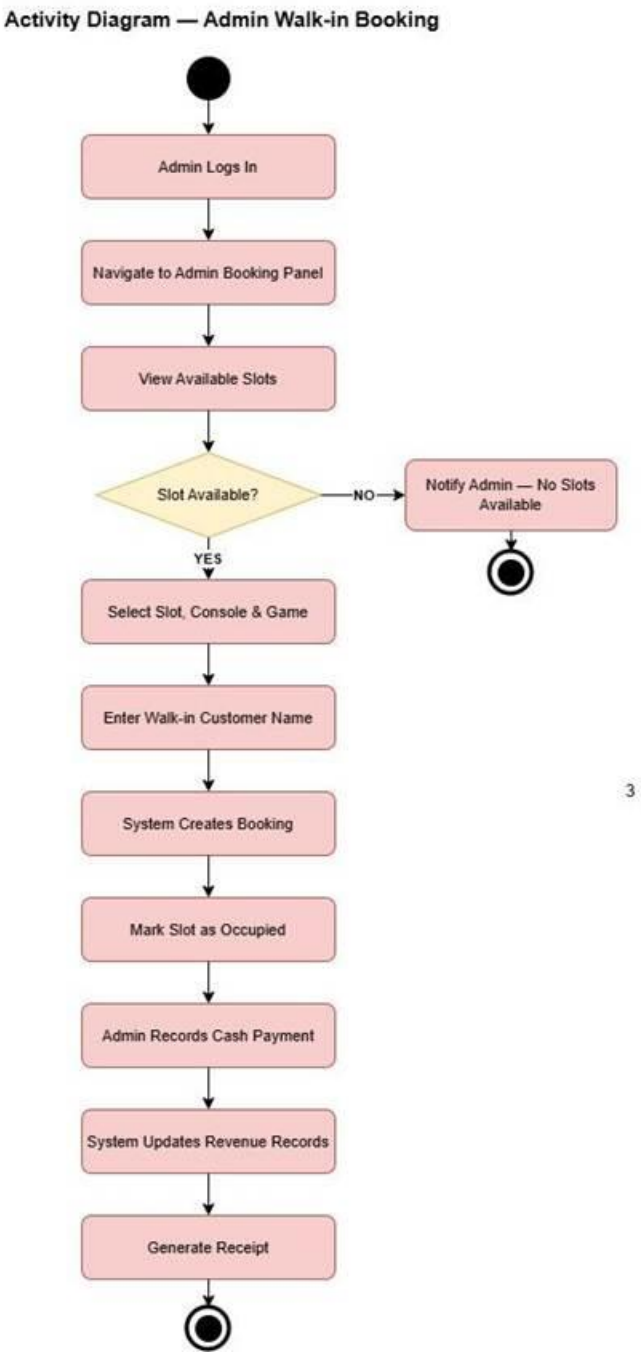


Figure B.4: Activity Diagram – Admin Walk-in Booking

B.7.4 Loyalty Points Redemption

This activity diagram describes the process by which a registered customer applies accumulated loyalty points as a discount during a slot booking payment.

Step	Activity	Actor	Decision / Outcome
1	Customer Logs In	Customer	–
2	Navigate to Book a Slot	Customer	–
3	View Loyalty Points Balance	System	–
4	Check Sufficient Points	System	Decision: Sufficient Points? NO → Proceed to full payment → END
5	Apply Loyalty Points Discount	System	–
6	View Discounted Booking Summary	Customer	–
7	Confirm & Pay Remaining Balance	Customer	–
8	Process Payment	System	Decision: Payment Successful? YES → Step 9 | NO → Restore points, notify → END
9	Deduct Redeemed Points from Balance	System	–
10	Booking Confirmed	System	Postcondition: Points balance updated

Table B.34: Activity Steps – Loyalty Points Redemption

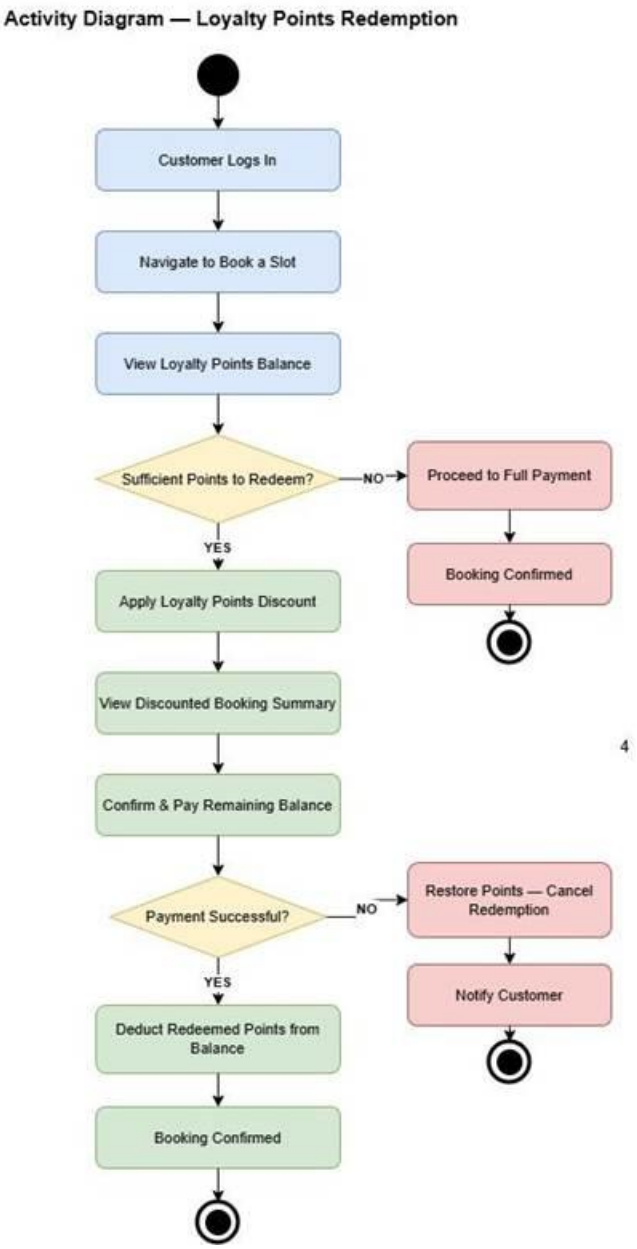


Figure B.5: Activity Diagram – Loyalty Points Redemption

B.7.5 User Login / Authentication

This activity diagram describes the authentication process for both customers and administrators accessing the Gaming Castle web system, including credential validation, role-based redirection, and account lockout after repeated failed attempts.

Step	Activity	Actor	Decision / Outcome
1	User Opens Login Page	User	–
2	Enter Email & Password	User	–
3	Submit Login Form	User	–
4	Validate Credentials	System	Decision: Credentials Valid? YES → Step 5 | NO → Show error
5	Check Failed Attempts	System	Decision: Attempts < 5? YES → Retry | NO → Lock account → END
6	Verify User Role	System	–
7	Role-based Redirect	System	Decision: Role? Customer → Customer Dashboard | Admin → Admin Dashboard
8	Generate & Store Session Token	System	Postcondition: User is authenticated and redirected

Table B.35: Activity Steps – User Login / Authentication

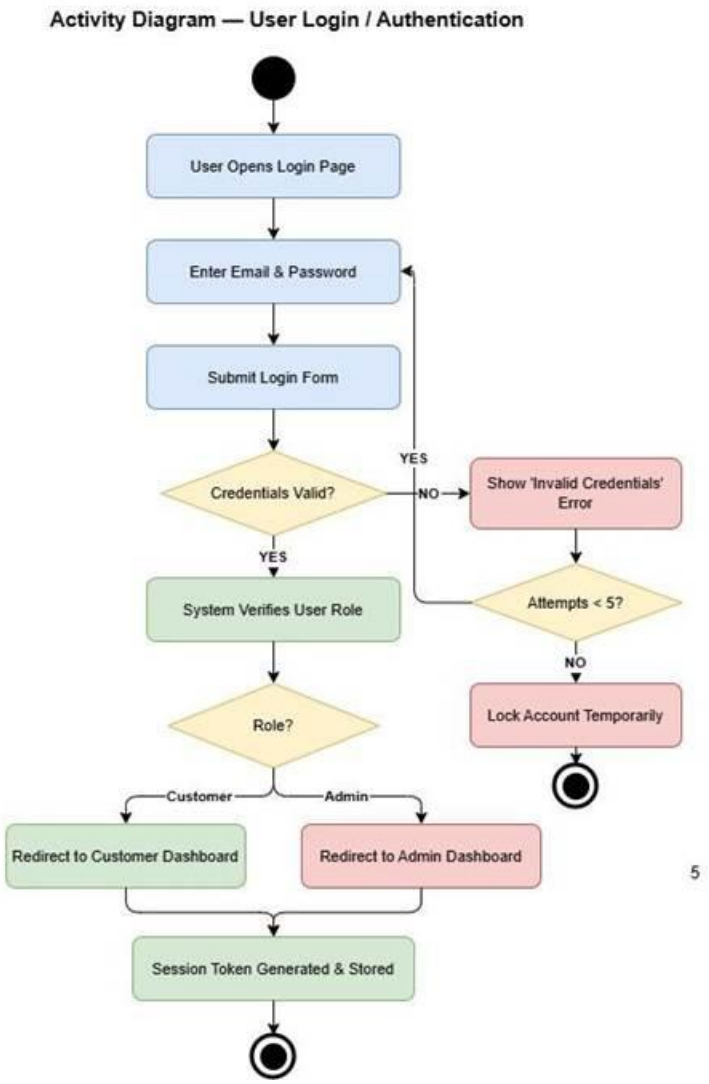


Figure B.6: Activity Diagram – User Login / Authentication

B.7.6 User Registration

This activity diagram describes the process by which a new user creates an account on the Gaming Castle web system, including form validation, duplicate email checking, account creation, and confirmation email dispatch.

Step	Activity	Actor	Decision / Outcome

1	User Opens Registration Page	User	–
2	Enter Name, Email, Phone & Password	User	–
3	Submit Registration Form	User	–
4	Validate All Fields	System	Decision: All Fields Valid? NO → Show inline errors → END
5	Check Email Uniqueness	System	Decision: Email Already Registered? YES → Show Email Exists error → END
6	Hash Password & Store Account	System	–
7	Assign Customer Role	System	–
8	Initialize Loyalty Points to 0	System	–
9	Send Confirmation Email	System	–
10	Redirect to Login Page	System	Postcondition: New customer account is active

Table B.36: Activity Steps – User Registration

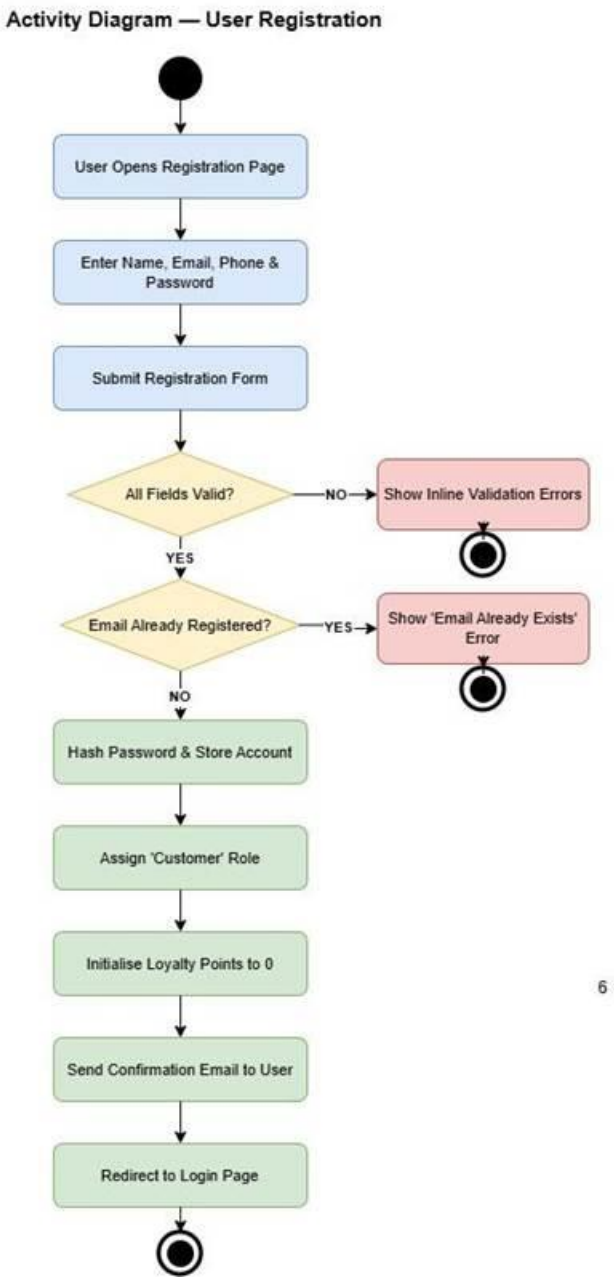


Figure B.7: Activity Diagram – User Registration

B.7.7 Payment Processing

This activity diagram describes the end-to-end payment processing workflow within the Gaming Castle system, covering optional loyalty point discount application, payment gateway interaction, receipt generation, and loyalty point award upon successful payment.

Step	Activity	Actor	Decision / Outcome
1	System Displays Payment Summary	System	–
2	Check Loyalty Points Option	System	Decision: Apply Loyalty Points? YES → Apply discount | NO → Proceed
3	Customer Selects Payment Method	Customer	–
4	Customer Enters Payment Details	Customer	–
5	Send Request to Payment Gateway	System	–
6	Await Gateway Response	System	Decision: Timeout? YES → Hold booking 10 min | Received → Step 7
7	Check Payment Approval	System	Decision: Approved? YES → Step 8 | NO → Show failure, release slot → END
8	Record Payment in Database	System	–
9	Generate & Display Digital Receipt	System	–
10	Award Loyalty Points to Customer	System	Postcondition: Payment confirmed, loyalty points updated

Table B.37: Activity Steps – Payment Processing

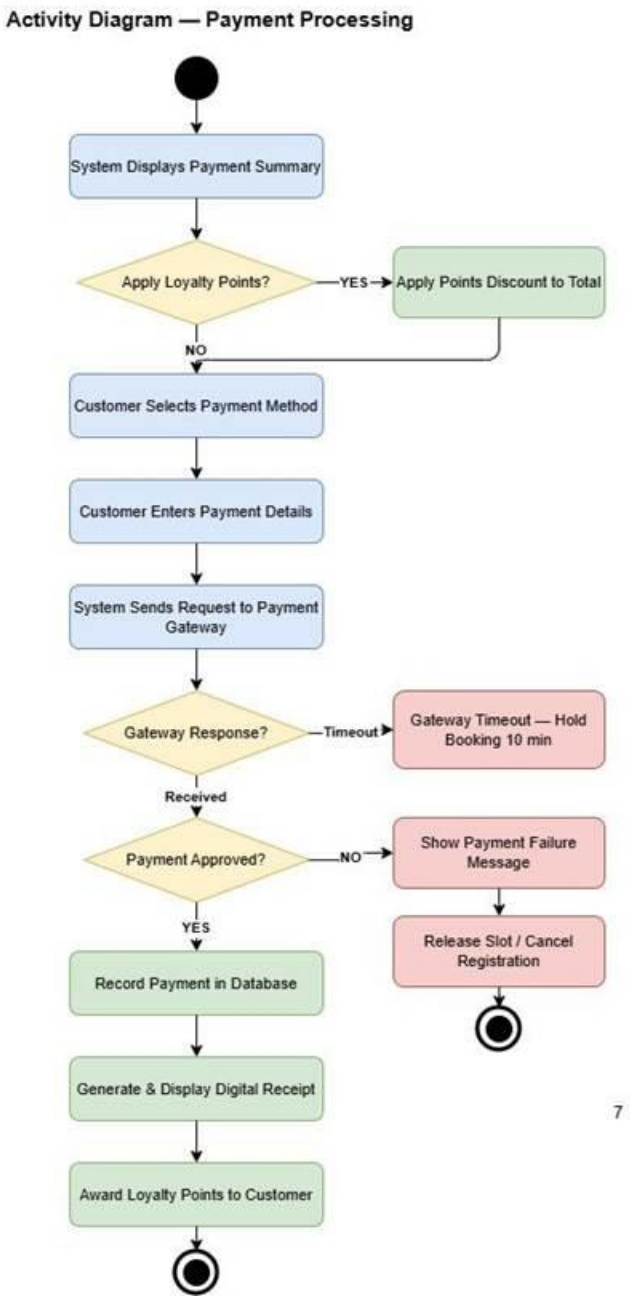


Figure B.8: Activity Diagram – Payment Processing

B.7.8 Admin Dashboard Overview

This activity diagram describes the workflow of an administrator interacting with the Gaming Castle admin dashboard, including navigation across key management modules: bookings, tournaments, users, and reports.

Step	Activity	Actor	Decision / Outcome
1	Admin Logs In Successfully	Admin	Precondition: Admin authenticated
2	System Loads Admin Dashboard	System	Displays: Real-time bookings, revenue summary, active sessions
3	Admin Selects Action	Admin	Decision: Manage Bookings | Manage Tournaments | Manage Users | Generate Reports
4a	Manage Bookings	Admin	View / cancel / create bookings
4b	Manage Tournaments	Admin	Create / update / cancel tournaments
4c	Manage Users	Admin	View / deactivate / edit accounts
4d	Generate Reports	Admin	Select report type and date range
5	Action Completed	System	All branches merge here
6	Continue on Dashboard?	Admin	Decision: YES → Refresh dashboard → Step 3 | NO → Logout → END

Table B.38: Activity Steps – Admin Dashboard Overview

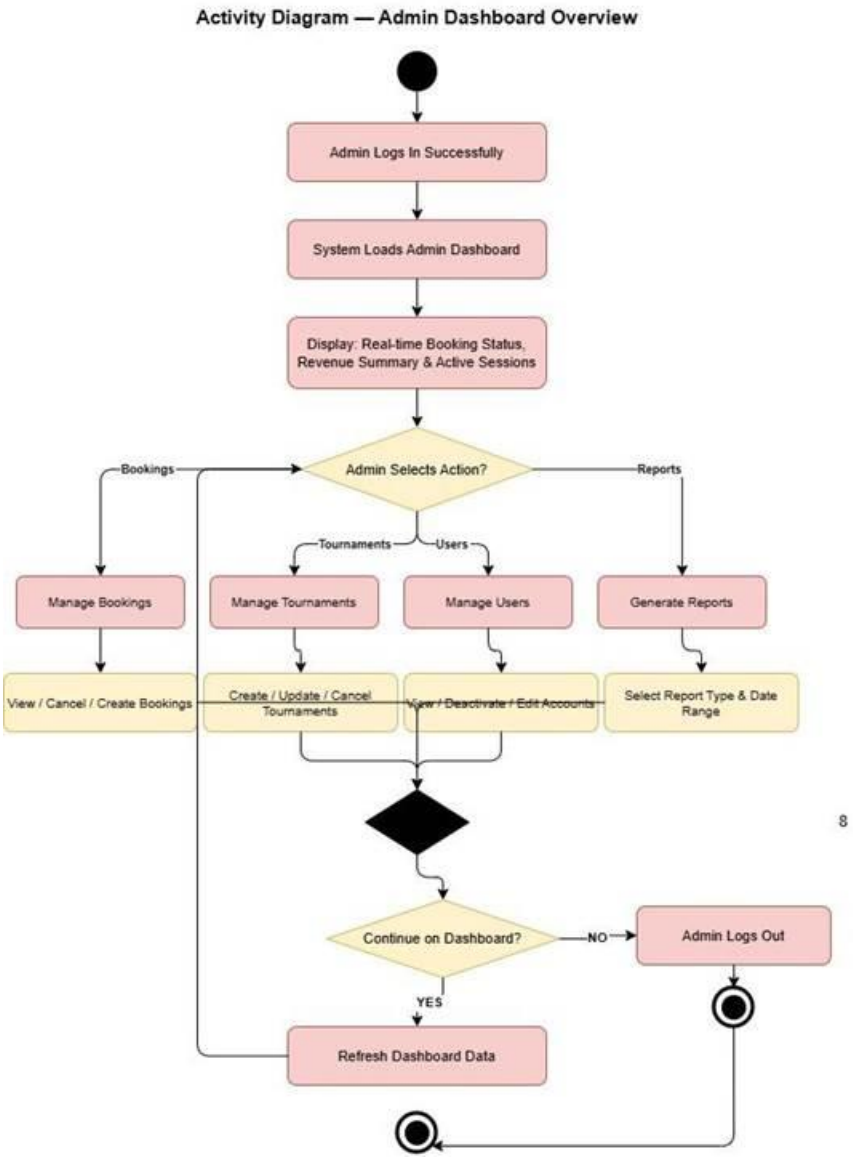


Figure B.9: Activity Diagram – Admin Dashboard Overview

B.8 External Interface Requirements

B.8.1 User Interface Requirements

A clean, intuitive, and responsive web-based user interface shall be provided by the system. The following requirements apply:

- The interface shall be accessible on desktop, tablet, and mobile devices through responsive design (minimum viewport width of 320px).

- The customer-facing interface shall include pages for: Home, Slot Booking, Tournaments, Loyalty Rewards, Profile, and Payment History.
- The admin interface shall include a dashboard with real-time booking status, revenue summary, tournament management, user management, and reporting tools.
- Error messages shall be displayed inline, written in plain English, and accompanied by suggested corrective actions.
- A consistent color scheme, typography, and iconography aligned with the Gaming Castle brand shall be used by the interface.

B.8.2 Hardware Interface Requirements

The Gaming Castle system is a web-based application and does not require direct integration with specialized hardware. However, the following hardware dependencies apply:

- A server or cloud hosting infrastructure with a minimum of 2 vCPU, 4 GB RAM, and 50 GB storage is required by the system.
- A modern web browser and stable internet connectivity are required at administrator terminals.
- No direct hardware-level integration with PS5 consoles is required; console allocation is managed logically within the system.

B.8.3 Software Interface Requirements

- Payment Gateway: A payment service provider shall be integrated via REST API for processing online payments.
- Email Service: An email service shall be integrated for sending booking confirmations, tournament notifications, and password reset emails.
- SMS Notifications: An SMS gateway may be integrated for booking confirmation messages.
- Database: A relational database management system shall be used for persistent data storage.
- Web Server: The system shall be served via a web application server compatible with SpringBoot.

B.8.4 Communications Interface Requirements

- All client-server communications shall use HTTPS over TLS 1.2 or higher.
- RESTful APIs shall be used for communication between the frontend and backend, with data exchanged in JSON format.
- The system shall operate over standard TCP/IP networks and shall not require any proprietary network protocols.
- WebSocket connections may be used for real-time slot availability updates.

B.8.5 System Architecture

The Gaming Castle system shall be designed using a microservices-based architecture to improve scalability, maintainability, and flexibility.

Each major module of the system, including User Management, Booking Management, Payment Processing, Tournament Management, and Loyalty Program, shall be implemented as independent microservices.

These microservices shall communicate with each other using RESTful APIs.

Docker containerization shall be used to package each microservice, ensuring consistent deployment across development, testing, and production environments.

An API Gateway shall be used to handle incoming client requests, manage routing to appropriate services, and enforce security mechanisms such as authentication and authorization.

This architecture provides the following benefits:

- Independent deployment of services
- Improved system scalability
- Easier maintenance and debugging
- Better fault isolation

B.9 Data Requirements

The following key data entities are required for the Gaming Castle system:

Entity	Key Attributes	Description

User	user_id, name, email, phone, password_hash, role, loyalty_points, created_at, updated_at	Stores registered customer and admin account information.
Console	console_id, name, type (PS5), console_status, games_available	Represents a physical PS5 console available for booking.
Booking	booking_id, user_id, console_id, date, start_time, end_time, total_charge, payment_status, booking_type, booking_status	Records all slot booking transactions, both online and walk-in.
Payment	payment_id, booking_id, amount, method (online/cash), payment_status, timestamp	Stores payment records linked to bookings.
Tournament	tournament_id, name, game, date, entry_fee, max_participants, status, created_by	Manages tournament events created by the administrator.
Tournament Registration	registration_id, tournament_id, user_id, payment_id, registered_at	Links customers to tournaments they have registered for.
Loyalty Transaction	transaction_id, user_id, points_awarded, points_redeemed, balance, booking_id, timestamp	Tracks loyalty point earn and redemption history.

Table B.39: Data Requirements – Key Entities

All personal data (names, emails, phone numbers) shall be stored in compliance with applicable data protection regulations. Customer passwords shall be stored exclusively as bcrypt hashes.

B.10 Alternative Solutions Considered

Alternative	Description	Reason for Rejection
WhatsApp / Phone-based Booking	Continuation with manual booking via phone calls or WhatsApp messages.	Does not scale; prone to human error; no automated payment or loyalty tracking; rejected in favor of the proposed web system.
Third-party Booking Platform	Adoption of an existing venue booking platform (e.g., Booksy, SimplyBook.me).	Generic platforms do not support gaming-specific features (tournament management, per-console booking, loyalty points); high recurring subscription costs.
Mobile Application (Native)	Development of a native iOS/Android mobile application instead of a web application.	Higher development cost and time; separate codebases are required for iOS and Android; a responsive web application achieves the same goals with a single codebase.
Spreadsheet-based System	Use of Google Sheets or Microsoft Excel for booking and payment tracking.	Not suitable for real-time multi-user access; lacks payment integration and automated notifications; not scalable.

Table B.40: Alternative Solutions Considered

B.11 Feasibility Study

B.11.1 Technical Feasibility

The proposed system is technically feasible. The required skills for building a web application using modern frameworks such as Angular.js (frontend) and Spring Boot (backend) are available within the development team. Relational databases (PostgreSQL), RESTful API design, and cloud hosting platforms are well-established technologies with extensive documentation and community support. Well-documented integration guides are provided by payment gateway APIs (Stripe). No novel or unproven technologies are required.

The use of Docker and microservices architecture further strengthens the technical feasibility of the system. Docker ensures a consistent runtime environment across different stages of development, while microservices enable modular development and scalability. The

development team has sufficient knowledge to implement and manage containerized services within the project scope.

An automated tournament management module can be implemented using rule-based logic or simple machine learning techniques. Given the scale of the system, a lightweight approach is sufficient to generate tournament brackets and manage match outcomes without requiring complex infrastructure.

B.11.2 Economic Feasibility

The system can be developed using open-source frameworks and tools at minimal licensing cost. Cost-effective deployment options are offered by cloud hosting providers (AWS Free Tier). The expected development cost is limited to the time investment of the four-member team within the academic project context. For Gaming Castle, a return on investment is projected to be generated by reducing manual overhead, increasing booking efficiency, and attracting a broader customer base through online accessibility and tournament events.

B.11.3 Operational Feasibility

The system is operationally feasible. The administrator interface is designed to be intuitive and requires minimal training. The customer-facing interface follows familiar web conventions for booking and payment. Internet connectivity is available at the Gaming Castle premises, and web-based services are routinely used by the target users. Customer engagement and retention are expected to be increased through the loyalty program and tournament features.

B.11.4 Schedule Feasibility

The project is planned to be completed within the academic semester. Four members with defined responsibilities constitute the team. The development is structured into sprints covering requirements analysis, system design, frontend development, backend development, integration testing, and final deployment. Based on the project scope and team capacity, the schedule is considered achievable within the allocated timeframe.

B.12 Risk Analysis

The following Risk Register documents identified project risks, their likelihood, impact, overall risk level, and proposed mitigation strategies.

Risk ID	Description	Category	Likelihood	Impact	Risk Level	Mitigation Strategy
R-01	Payment gateway integration failure or delays	Technical	Medium	High	High	Multiple payment gateway options shall be researched early; a mock payment module shall be developed for testing.
R-02	Scope creep due to evolving client requirements	Requirements	High	Medium	High	A formal change request process shall be established; client sign-off on requirements shall be obtained before development.

R-03	Team member unavailability due to illness or academic workload	Resource	Medium	Medium	Medium	All work shall be thoroughly documented; knowledge shall be distributed across the team; task backups shall be maintained.
R-04	Security vulnerabilities in the web application	Technical	Medium	High	High	An OWASP Top 10 assessment shall be conducted; HTTPS and input validation shall be enforced; peer code reviews shall be performed.
R-05	Project timeline overrun	Schedule	Medium	High	High	Agile sprint planning shall be used; weekly progress reviews shall be conducted; core features shall be prioritized for MVP delivery.

R-06	Low user adoption post-deployment	External	Low	Medium	Low	The client and target users shall be involved in UAT; a user guide and onboarding support shall be provided.
R-07	Third-party API deprecation or changes	Technical	Low	High	Medium	Stable, well-maintained APIs shall be selected; abstraction layers shall be implemented to facilitate API replacement.
R-08	Data loss due to server failure	Technical	Low	High	Medium	Automated daily database backups with off-site storage shall be implemented; recovery procedures shall be tested.

Table B.41: Risk Register

B.13 References

[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[2] ISO/IEC, "Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models," ISO/IEC 25010:2011, 2011.

[3] OWASP Foundation, "OWASP Top Ten," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. (accessed Mar. 11, 2026).

[4] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. (accessed Mar. 11, 2026).

Client Requirement Sign-Off

The client and project supervisor must review and approve the scope, requirements, and features documented in this SRS before the project proceeds to the design phase. Any changes to the approved requirements after this sign-off must be submitted as a formal Change Request and re-approved by both parties.

Client Representative Sign-Off
Name: Lathurshigan
Title: Owner
Organization: Gaming Castle
Date: 16/04/2026

Signature: 

Project Supervisor Sign-Off
Name: _____

Date: _____

Signature: _____

Chapter C System Design Specification (SDS)

This chapter presents the System Design Specification for the Gaming Castle Online Gaming Centre Management System. It translates the approved requirements from Chapter 2 into a comprehensive technical design, covering architectural design, database design, class diagram, sequence diagrams, UI/UX design, security design, and deployment design. Each section provides justification for design decisions taken.

C.1 Introduction

C.1.1 Purpose

This System Design Specification (SDS) document defines the technical design decisions for the Gaming Castle Online Gaming Centre Management System. This document serves as the primary technical reference for the development team and describes the system architecture, database design, class structure, sequence diagrams, UI/UX design, security design, and deployment strategy.

The intended audience of this document includes the development team, academic evaluators, and the client. This document is to be read in conjunction with the Software Requirements Specification (SRS) for Gaming Castle.

C.1.2 Scope

The Gaming Castle system is a web-based platform designed to manage slot bookings, tournament registrations, payment processing, loyalty rewards, and administrative operations for a PS5 gaming hub in Nelliyadi, Sri Lanka. This SDS document provides the complete technical design required to implement the system as defined in the SRS.

C.1.3 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
SDS	System Design Specification
SRS	Software Requirements Specification
MVC	Model-View-Controller architectural pattern
API	Application Programming Interface
ER Diagram	Entity-Relationship Diagram
JWT	JSON Web Token
RBAC	Role-Based Access Control
CI/CD	Continuous Integration / Continuous Deployment
WCAG	Web Content Accessibility Guidelines

TLS	Transport Layer Security
DXA	Document Exchange Architecture (unit of measurement in Word)
VPC	Virtual Private Cloud
CDN	Content Delivery Network
SPA	Single Page Application

Table C.1: Definitions, Acronyms and Abbreviations

C.1.4 Document Overview

This document is organized as follows:

- Chapter 1 – Introduction: Provides the purpose, scope, definitions, and overview.
- Chapter 2 – Architectural Design: Describes the high-level architecture and justification.
- Chapter 3 – Database Design: Presents the ER diagram and normalization.
- Chapter 4 – Class Diagram: Details classes, attributes, methods, and relationships.
- Chapter 5 – Sequence Diagrams: Illustrates interaction flows for all major use cases.
- Chapter 6 – UI/UX Design: Covers wireframes, user flow, and accessibility.
- Chapter 7 – Security Design: Defines authentication, authorization, and data protection.
- Chapter 8 – Deployment Design: Specifies the hosting plan, CI/CD pipeline, and environments.
- Chapter 9 – References: Lists all cited sources and standards.

C.2 Architectural Design

C.2.1 High-Level Architecture

The Gaming Castle system shall be implemented using a Microservices-based Architecture with a layered internal structure within each service. The system comprises five independently deployable microservices, each responsible for a distinct business domain.

The five core microservices are:

- User Service – Manages customer and administrator account registration, authentication, and profile management.
- Booking Service – Handles PS5 slot availability, online bookings, walk-in bookings, and cancellation or rescheduling.
- Payment Service – Processes online payments via a payment gateway and records cash payments made by the administrator.
- Tournament Service – Manages tournament creation, participant registration, bracket generation, and status updates.
- Loyalty Service – Tracks loyalty point accrual, redemption, and balance queries for registered customers.

Each microservice internally follows the Model-View-Controller (MVC) layered pattern with a defined separation of concerns across the Controller layer (REST API endpoints), Service layer (business logic), Repository layer (database access), and Model layer (data entities).

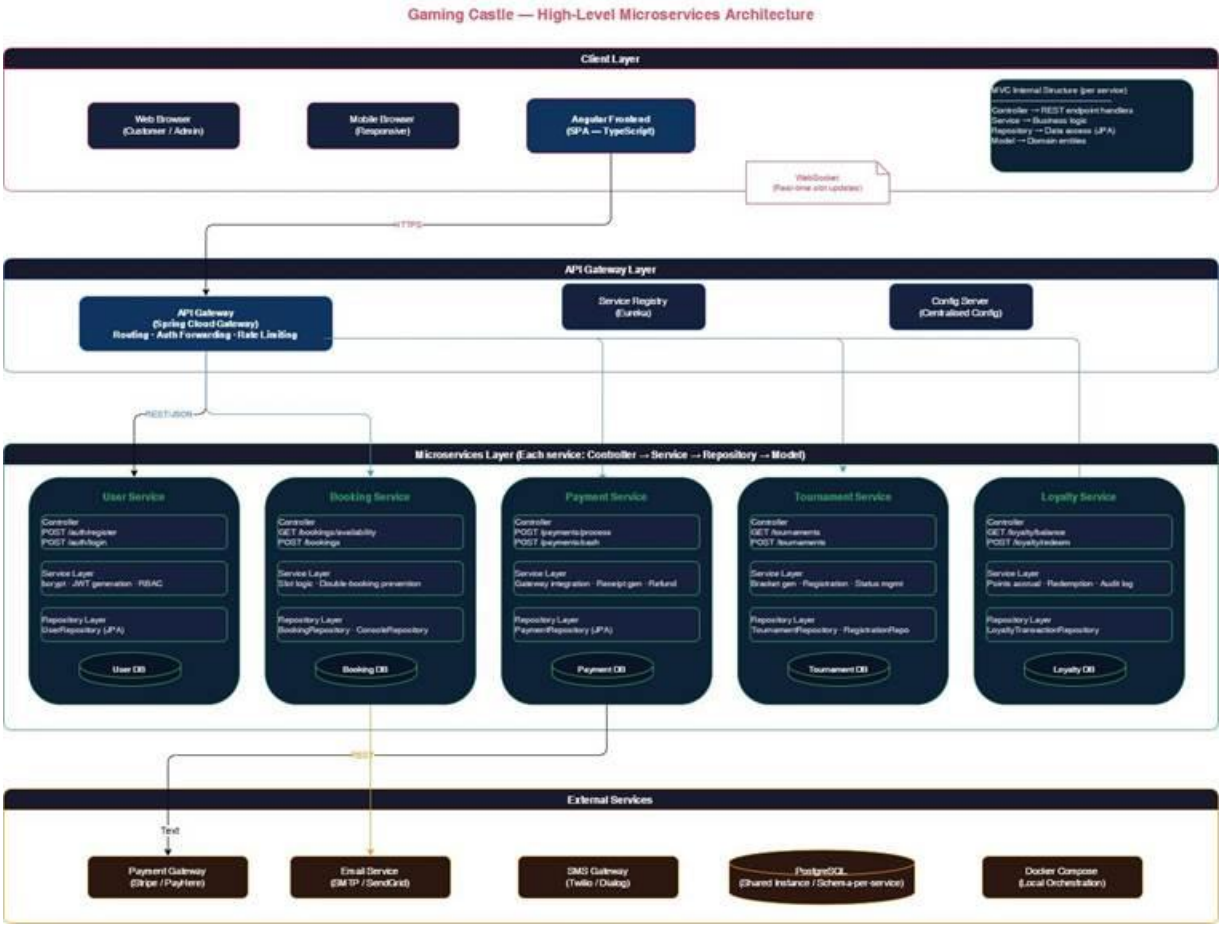


Figure C.1: High-Level Architecture Diagram

C.2.2 Cloud-Native Architecture

The system is designed as a cloud-native application to ensure scalability, resilience, and ease of deployment. The following cloud-native design principles are applied:

- Containerization: Each microservice shall be packaged as a Docker container, ensuring environment consistency across development, staging, and production.
- API Gateway: An API Gateway (e.g., Spring Cloud Gateway) shall serve as the single entry point for all client requests, handling routing, authentication forwarding, and rate limiting.
- Service Registry: A service registry (e.g., Eureka) shall be used for microservice discovery, enabling dynamic routing and fault tolerance.

- Centralized Configuration: A configuration server shall manage environment-specific configuration across all microservices.
- Horizontal Scaling: Each service may be scaled independently by adding container instances behind a load balancer, supporting the NFR requirement of a 3x user load increase without redesign.

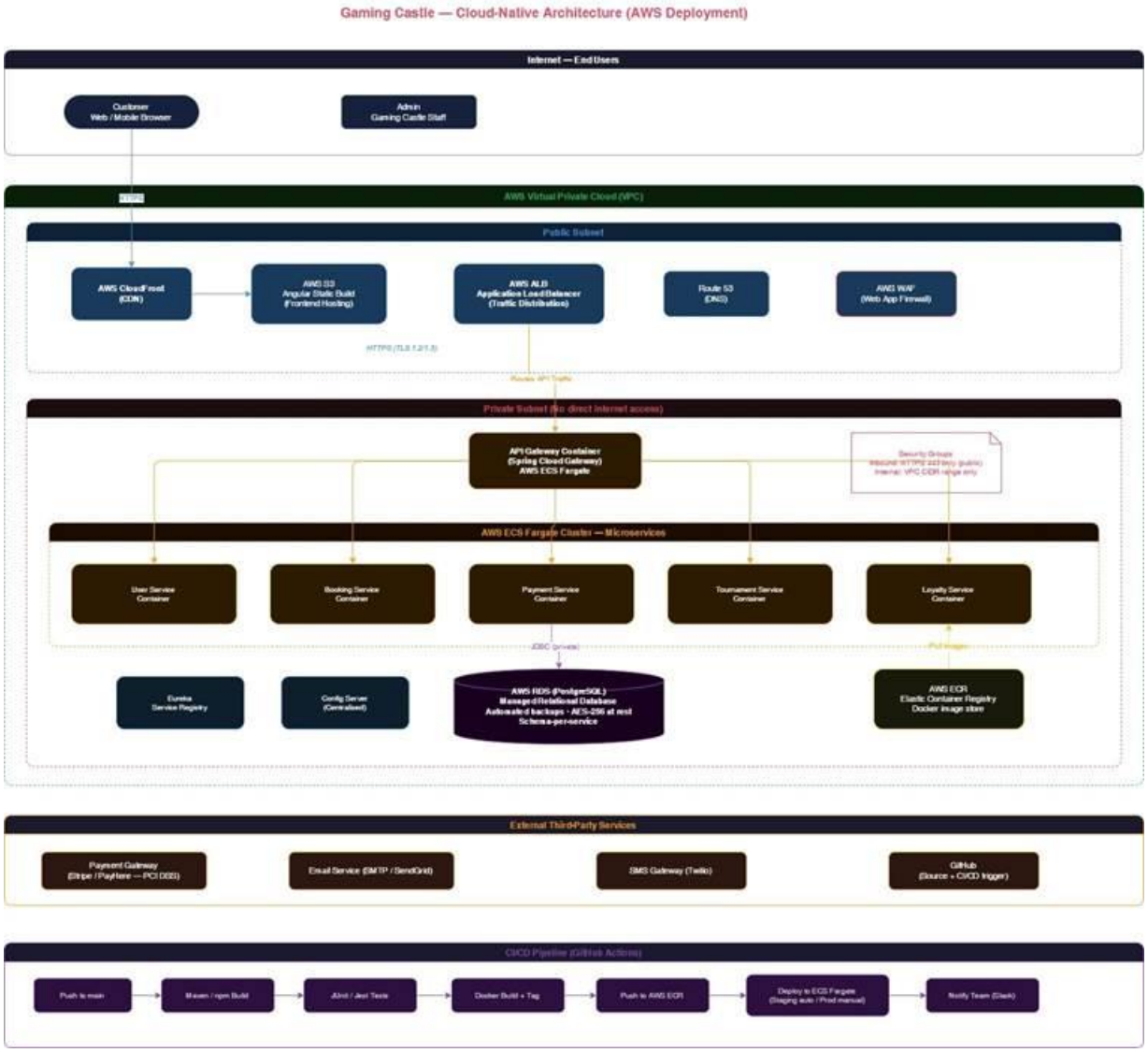


Figure C.2: Cloud-Native Architecture

C.2.3 Justification for Architecture Choice

The Microservices architecture was selected over monolithic and layered alternatives for the following reasons:

Criterion	Microservices (Selected)	Monolithic (Rejected)
Independent Deployment	Each service deployed independently	Full redeployment required
Scalability	Per-service horizontal scaling	Scale entire application
Fault Isolation	Failure in one service does not cascade	Single point of failure
Team Alignment	Each member owns a domain service	Shared codebase conflicts
Maintainability	Small, focused, testable services	Large codebase increases complexity

Table C.2: Architecture Comparison

C.2.4 Scalability and Maintainability Considerations

- Each microservice may be independently upgraded, replaced, or scaled without affecting others.
- Docker Compose shall be used for local development orchestration; Kubernetes or AWS ECS may be introduced for production deployment.

- A shared database-per-service pattern shall be adopted, ensuring loose coupling between data stores.
- All inter-service communication shall be conducted via RESTful HTTP APIs using JSON payloads.
- WebSocket connections shall be supported for real-time slot availability updates from the Booking Service to the frontend.

C.2.5 Technology Stack

- Frontend: Angular
- Backend: Spring Boot (Microservices)
- Database: PostgreSQL
- Authentication: JWT
- Containerization: Docker
- CI/CD: GitHub Actions
- Cloud: AWS (ECS, RDS, S3)

C.3 Database Design

C.3.1 Overview

A relational database management system (PostgreSQL) shall be used as the primary data store for the Gaming Castle system. A database-per-service strategy is adopted for the microservices architecture. Each service maintains its own schema within a shared PostgreSQL instance, allowing independent data management while avoiding the operational complexity of fully separate database servers in the initial deployment.

C.3.2 Entity-Relationship Diagram

The following key entities and relationships constitute the data model for the Gaming Castle system. The ER diagram below illustrates the full relational structure, cardinalities, and foreign key relationships.

Entity	Primary Key	Foreign Keys / Relationships

User	user_id	None (root entity)
Console	console_id	None (managed by admin)
Booking	booking_id	user_id → User, console_id → Console
Payment	payment_id	booking_id → Booking
Tournament	tournament_id	created_by → User (Admin)
TournamentRegistration	registration_id	tournament_id → Tournament, user_id → User, payment_id → Payment
LoyaltyTransaction	transaction_id	user_id → User, booking_id → Booking
Review	review_id	user_id → User, booking_id → Booking
Game	game_id	Console_id→ Console
GameConsole	game_id,console_id	game_id→Game, console_id→Console

Table C.3: Entity-Relationship Summary

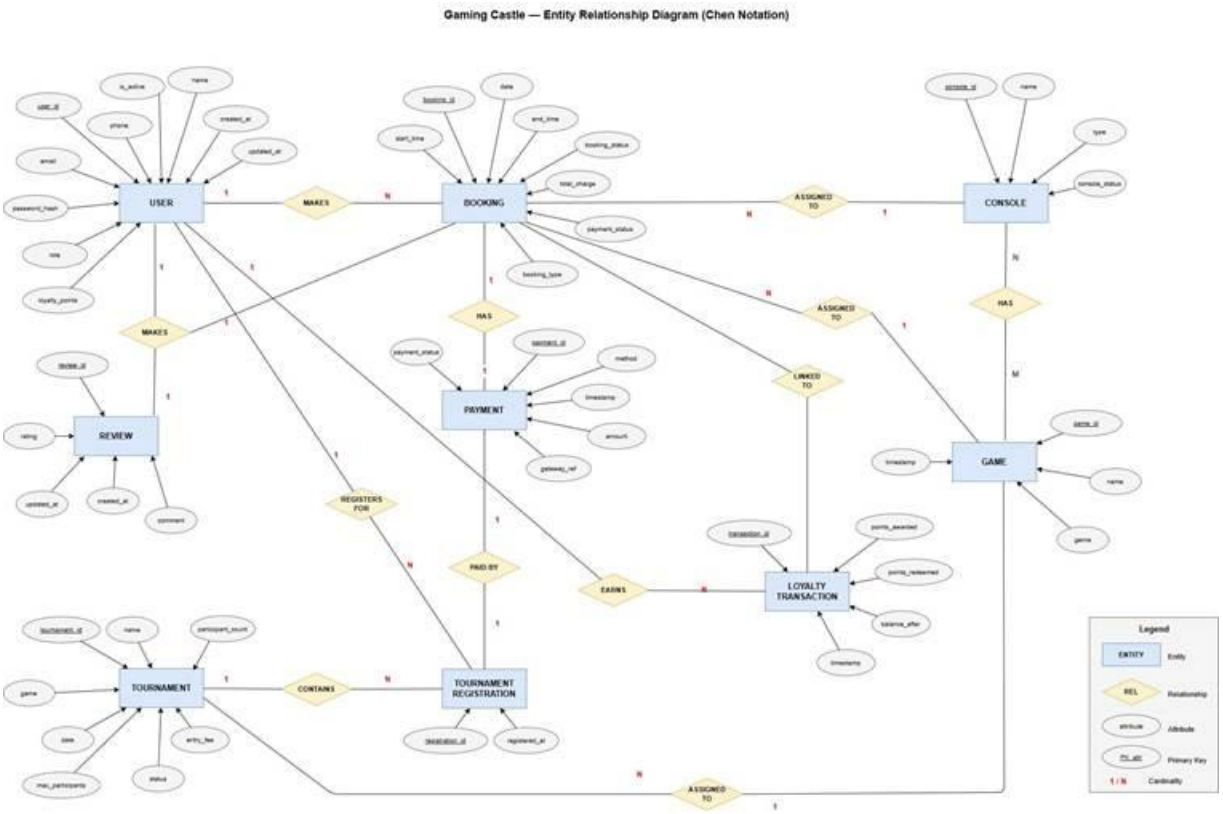


Figure C.3: Entity-Relationship Diagram

C.3.3 Key Entity Attributes

Entity	Attributes	Notes
User	user_id, name, email, phone, password_hash, role (CUSTOMER/ADMIN), loyalty_points, is_active, created_at, updated_at	bcrypt hashed password; role enum

Console	console_id, name, type (PS5), console_status (AVAILABLE/OCCUPIED/MAINTENANCE)	Status updated on booking
Booking	booking_id, user_id, console_id, date, game_id, start_time, end_time, total_charge, payment_status, booking_type (ONLINE/WALKIN), booking_status (CONFIRMED/CANCELLED)	Double-booking prevented by unique constraint
Payment	payment_id, booking_id, amount, method (ONLINE/CASH), payment_status, gateway_ref, timestamp	gateway_ref null for cash
Tournament	tournament_id,game_id, name, game, date, entry_fee, max_participants, participant_count, status (UPCOMING/ONGOING/COMPLETED/CANCELLED), created_by	Auto-closes registration when full
TournamentRegistration	registration_id, tournament_id, user_id, payment_id, registered_at	Unique(tournament_id, user_id)
Loyalty Transaction	transaction_id, user_id, points_awarded, points_redeemed, balance_after, booking_id, timestamp	Full audit trail of points history
Review	review_id,booking_id, user_id,rating,comment,created_at,updated_at	After complete game user can rating and review

Game	game_id,name,genre,created_at, updated_at	While booking show game available on that console.
Console Game	game_id,console_id,created_at,updated_at	Create a game by admin in particular console.

Table C.4: Key Entity Attributes

C.3.4 Normalisation

The database schema is designed to conform to the Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity. The following principles are applied:

- First Normal Form (1NF): All attributes contain atomic values. No repeating groups are used. Each table has a primary key.
- Second Normal Form (2NF): All non-key attributes are fully functionally dependent on the entire primary key. No partial dependencies exist in any table.
- Third Normal Form (3NF): All non-key attributes depend directly on the primary key and not on other non-key attributes. Transitive dependencies are eliminated. For example, customer name and email are stored only in the User table and referenced via foreign keys in Booking and TournamentRegistration.
- Indexes: Indexes shall be applied on frequently queried foreign keys (user_id, console_id, booking_id) and on date/time columns in the Booking table to support performant calendar queries.

C.4 Class Diagram

C.4.1 Overview

The class diagram below defines the main domain model classes for the Gaming Castle system. The classes correspond to the data entities and service-layer components identified in the architectural and database design. Each class includes its attributes (with visibility and type), methods, and relationships.

C.4.2 Core Domain Classes

Class	Key Attributes	Key Methods
User	- userId: UUID - name: String - email: String - phone: String - passwordHash: String - role: Role - loyaltyPoints: int - isActive: boolean	+ register(): void + login(email, password): Token + resetPassword(token, newPwd): void + updateProfile(details): void + deactivate(): void
Console	- consoleId: UUID - name: String - type: String - status: ConsoleStatus - games: List<Game>	+ getAvailableSlots(date): List<TimeSlot> + markOccupied(slot): void + markAvailable(slot): void

Booking	- bookingId: UUID - userId: UUID - gameId: UUID - consoleId: UUID - date: LocalDate - startTime: LocalTime - endTime: LocalTime - totalCharge: BigDecimal - paymentStatus: PaymentStatus - bookingType: BookingType - bookingStatus: BookingStatus	+ create(details): Booking + cancel(): void + reschedule(newSlot): void + calculateCharge(): BigDecimal + confirmPayment(): void
Payment	- paymentId: UUID - bookingId: UUID - amount: BigDecimal - method: PaymentMethod - status: PaymentStatus - gatewayRef: String - timestamp: DateTime	+ processOnlinePayment(): void + recordCashPayment(): void + generateReceipt(): Receipt + refund(): void
Tournament	- tournamentId: UUID - name: String - gameId: UUID - date: LocalDate - entryFee: BigDecimal - maxParticipants: int - participantCount: int - status: TournamentStatus - createdBy: UUID	+ create(details): Tournament + publish(): void + cancel(): void + updateStatus(status): void + generateBracket(): Bracket + isRegistrationOpen(): boolean

TournamentRegistration	- registrationId: UUID - tournamentId: UUID - userId: UUID - paymentId: UUID - registeredAt: DateTime	+ register(user, tournament): void + cancelRegistration(): void + notifyParticipant(): void
LoyaltyTransaction	- transactionId: UUID - userId: UUID - pointsAwarded: int - pointsRedeemed: int - balanceAfter: int - bookingId: UUID - timestamp: DateTime	+ awardPoints(booking): void + redeemPoints(amount): void + getBalance(userId): int + getHistory(userId): List<LoyaltyTransaction>
Review	- reviewId: UUID - bookingId: UUID - userId: UUID - rating: int - comment: String - createdAt: DateTime - updatedAt: DateTime	+ addReview(bookingId, userId, rating, comment): Review + updateReview(reviewId, rating, comment): void + deleteReview(reviewId): void + getReviewByBooking(bookingId): Review + getUserReviews(userId): List<Review>
Game	- gameId: UUID - name: String - genre: Genre - createdAt: DateTime - updatedAt: DateTime	+ create(): void, + update(): void, + delete(): void, + getByConsole(consoleId): List<Game>

Table C.5: Core Domain Classes

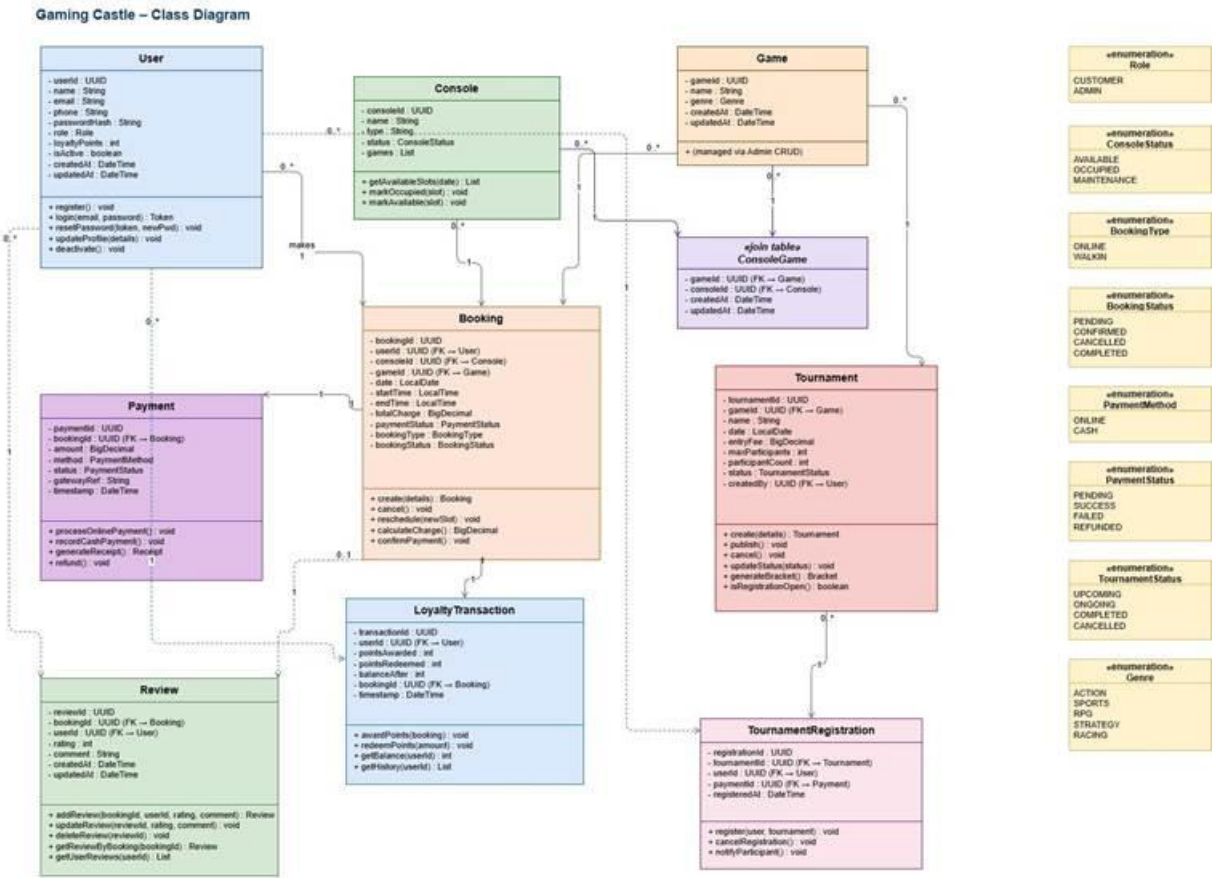


Figure C.4: Class Diagram

C.4.3 Relationships

- User – Booking: One-to-Many. A single user may hold multiple bookings.
- Console – Booking: One-to-Many. A single console may be associated with multiple bookings across different time slots.
- Console – Game: Many-to-Many. A console can support multiple games, and a game can be available on multiple consoles.
- Game – Booking: One-to-Many. A single game may be associated with multiple bookings, while each booking is for one selected game.
- Booking – Review: One-to-One. Each booking can have at most one review.
- User – Review: One-to-Many. A user may write multiple reviews for different bookings.

- Booking – Payment: One-to-One. Each booking is associated with exactly one payment record.
- User – TournamentRegistration: One-to-Many. A user may register for multiple tournaments.
- Tournament – TournamentRegistration: One-to-Many. A tournament may have multiple registered participants.
- Game – Tournament: One-to-Many. A single game can have multiple tournaments, while each tournament is associated with one game.
- Booking – LoyaltyTransaction: One-to-One. Each completed booking generates one loyalty transaction record.

C.4.4 Enumerations

Enumeration	Values
Role	CUSTOMER, ADMIN
ConsoleStatus	AVAILABLE, OCCUPIED, MAINTENANCE
BookingType	ONLINE, WALKIN
BookingStatus	PENDING, CONFIRMED, CANCELLED, COMPLETED
PaymentMethod	ONLINE, CASH
PaymentStatus	PENDING,SUCCESS, FAILED, REFUNDED

TournamentStatus	UPCOMING, ONGOING, COMPLETED, CANCELLED
Genre	ACTION,SPORTS,RPG,STRATEGY,RACING

Table C.6: Domain Enumerations

C.5 Sequence Diagrams

C.5.1 Overview

This chapter presents sequence diagrams for all major use cases of the Gaming Castle system. Each diagram describes the temporal interaction between actors and system components, including the frontend (Angular), API Gateway, relevant microservice, and the database.

C.5.2 UC-01: Register Account

The registration sequence involves client-side input submission, server-side field validation, uniqueness checking, password hashing, account creation, and confirmation email dispatch.

Step	From	To	Message / Action
1	Customer (Browser)	Angular Frontend	Submit registration form (name, email, phone, password)
2	Angular Frontend	API Gateway	POST /api/auth/register
3	API Gateway	User Service	Forward registration request

4	User Service	User DB	SELECT: check email uniqueness
5	User Service	User Service	Hash password using bcrypt
6	User Service	User DB	INSERT: new user record, role=CUSTOMER, loyalty_points=0
7	User Service	Email Service	Send confirmation email
8	User Service	Angular Frontend	201 Created – Redirect to Login page

Table C.7: Sequence – UC-01 Register Account

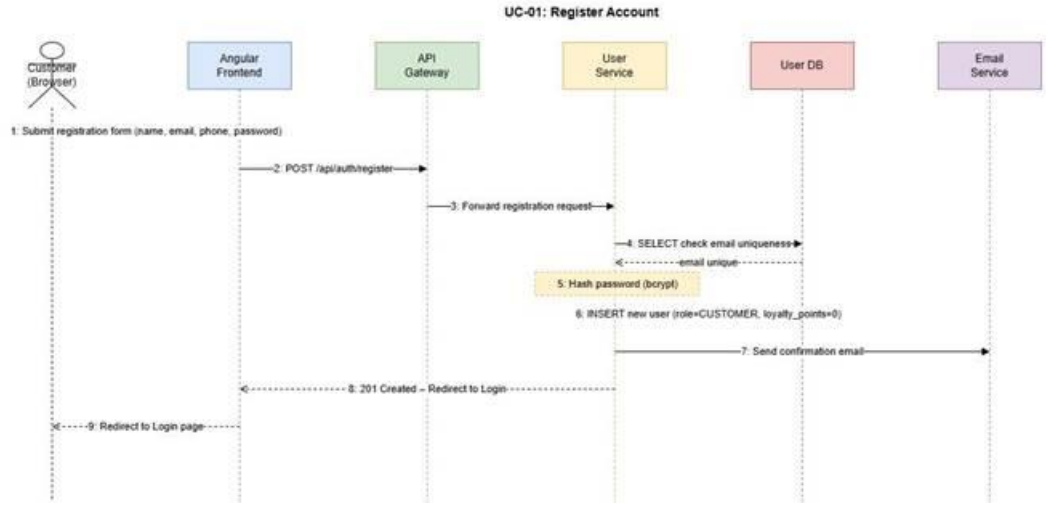


Figure C.5: Sequence – UC-01 Register Account

C.5.3 UC-02: Login to System

Step	From	To	Message / Action
1	User (Browser)	Angular Frontend	Submit login form (email, password)
2	Angular Frontend	API Gateway	POST /api/auth/login
3	API Gateway	User Service	Forward login request
4	User Service	User DB	SELECT: fetch user by email; validate bcrypt hash
5	User Service	User Service	Check failed_attempts; lock account if >= 5
6	User Service	User Service	Determine role (CUSTOMER or ADMIN); generate JWT
7	User Service	Angular Frontend	200 OK – return JWT token + role
8	Angular Frontend	Browser	Redirect to Customer Dashboard or Admin Dashboard based on role

Table C.8: Sequence – UC-02 Login to System

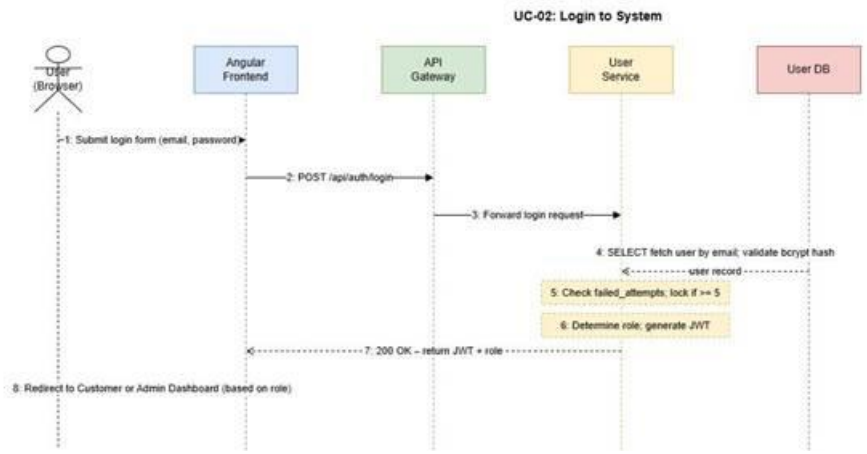


Figure C.6: Sequence – UC-02 Login to System

C.5.4 UC-03: Book Gaming Slot

Step	From	To	Message / Action
1	Customer	Angular Frontend	Navigate to Book a Slot; select date, time, console, game
2	Angular Frontend	Booking Service	GET /api/bookings/availability?date=X&time=Y
3	Booking Service	Booking DB	SELECT: check no existing confirmed booking for console/slot
4	Booking Service	Angular Frontend	Return availability + booking summary with charge

5	Customer	Angular Frontend	Confirm booking; proceed to payment
6	Angular Frontend	Payment Service	POST /api/payments/process (bookingDetails, paymentMethod)
7	Payment Service	Payment Gateway	Send payment request; await gateway response
8	Payment Service	Booking Service	Payment confirmed: update booking status to CONFIRMED
9	Booking Service	Loyalty Service	Award loyalty points for completed booking
10	Booking Service	Notification Service	Send booking confirmation via email/SMS

Table C.9: Sequence – UC-03 Book Gaming Slot

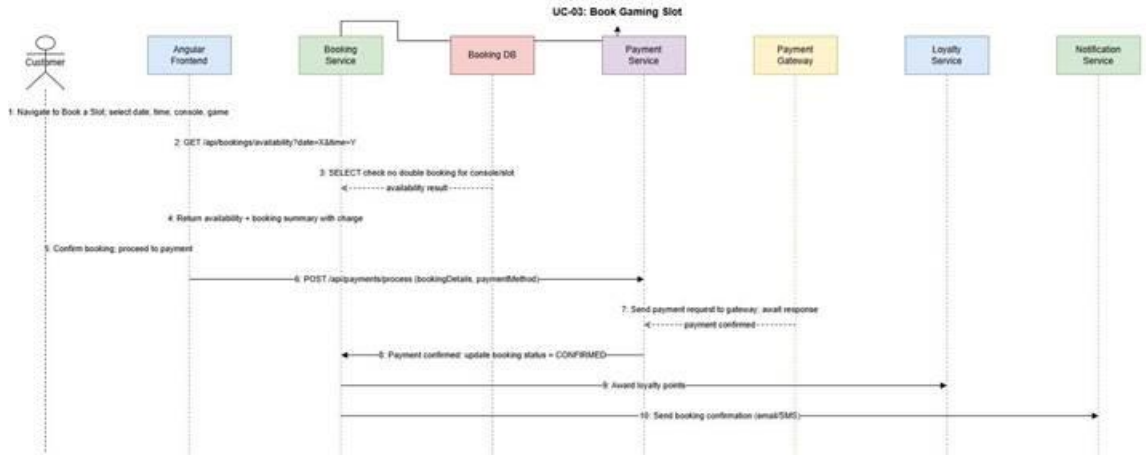


Figure C.7: Sequence – UC-03 Book Gaming Slot

C.5.5 UC-05: Admin Book Slot for Walk-in Customer

Step	From	To	Message / Action
1	Admin	Angular Frontend	Navigate to Admin Booking panel; select slot, console, game
2	Angular Frontend	Booking Service	GET /api/admin/bookings/availability
3	Admin	Angular Frontend	Enter walk-in customer name; submit booking
4	Angular Frontend	Booking Service	POST /api/admin/bookings (booking details, type=WALKIN)
5	Booking Service	Booking DB	INSERT: booking record; mark slot as OCCUPIED
6	Admin	Angular Frontend	Record cash payment: enter amount received
7	Angular Frontend	Payment Service	POST /api/payments/cash (bookingId, amount)

8	Payment Service	Payment DB	INSERT: payment record (method=CASH); update revenue records
9	Payment Service	Angular Frontend	Generate and display receipt

Table C.10: Sequence – UC-05 Admin Walk-in Booking

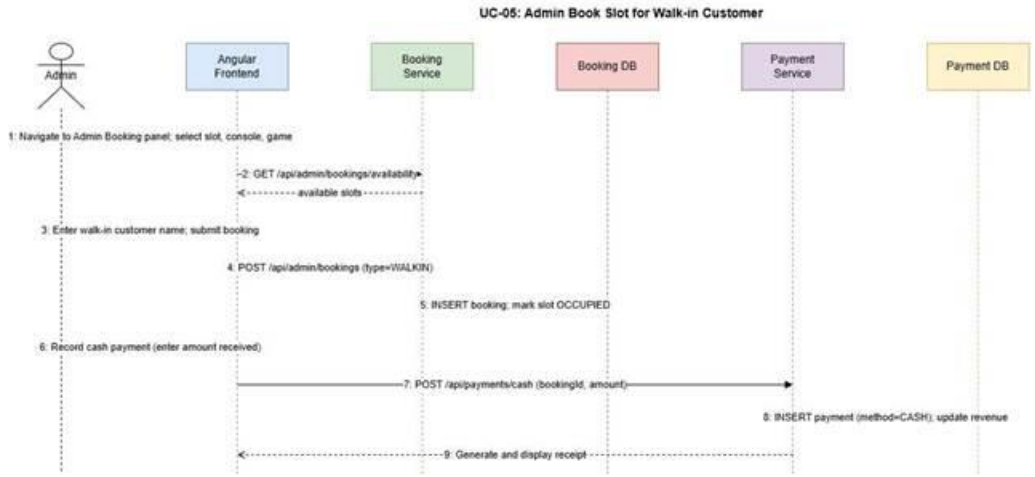


Figure C.8: Sequence – UC-05 Admin Walk-in Booking

C.5.6 UC-08: Register for Tournament

Step	From	To	Message / Action
1	Customer	Angular Frontend	Navigate to Tournaments page; view available tournaments

2	Angular Frontend	Tournament Service	GET /api/tournaments?status=UPCOMING
3	Customer	Angular Frontend	Select tournament; click Register
4	Tournament Service	Tournament DB	Check: registration open? participant_count < max_participants?
5	Angular Frontend	Payment Service	POST /api/payments/process (entry fee)
6	Payment Service	Tournament Service	Payment confirmed: register participant; increment participant_count
7	Tournament Service	Notification Service	Send confirmation notification to customer

Table C.11: Sequence – UC-08 Register for Tournament

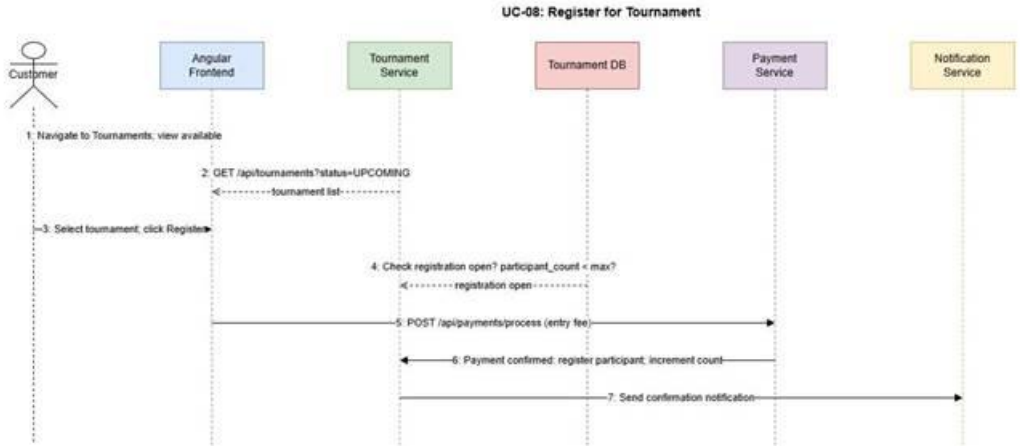


Figure C.9: Sequence – UC-08 Register for Tournament

C.5.7 UC-11: Redeem Loyalty Points

Step	From	To	Message / Action
1	Customer	Angular Frontend	Proceed to payment for a booking
2	Angular Frontend	Loyalty Service	GET /api/loyalty/balance (userId)
3	Angular Frontend	Customer	Display available points and equivalent discount
4	Customer	Angular Frontend	Select 'Redeem Points'; view discounted total
5	Angular Frontend	Payment Service	POST /api/payments/process (amount after discount, loyaltyPointsUsed)
6	Payment Service	Loyalty Service	Deduct redeemed points from balance; log LoyaltyTransaction

7	Payment Service	Angular Frontend	Booking confirmed; updated loyalty balance displayed

Table C.12: Sequence – UC-11 Redeem Loyalty Points

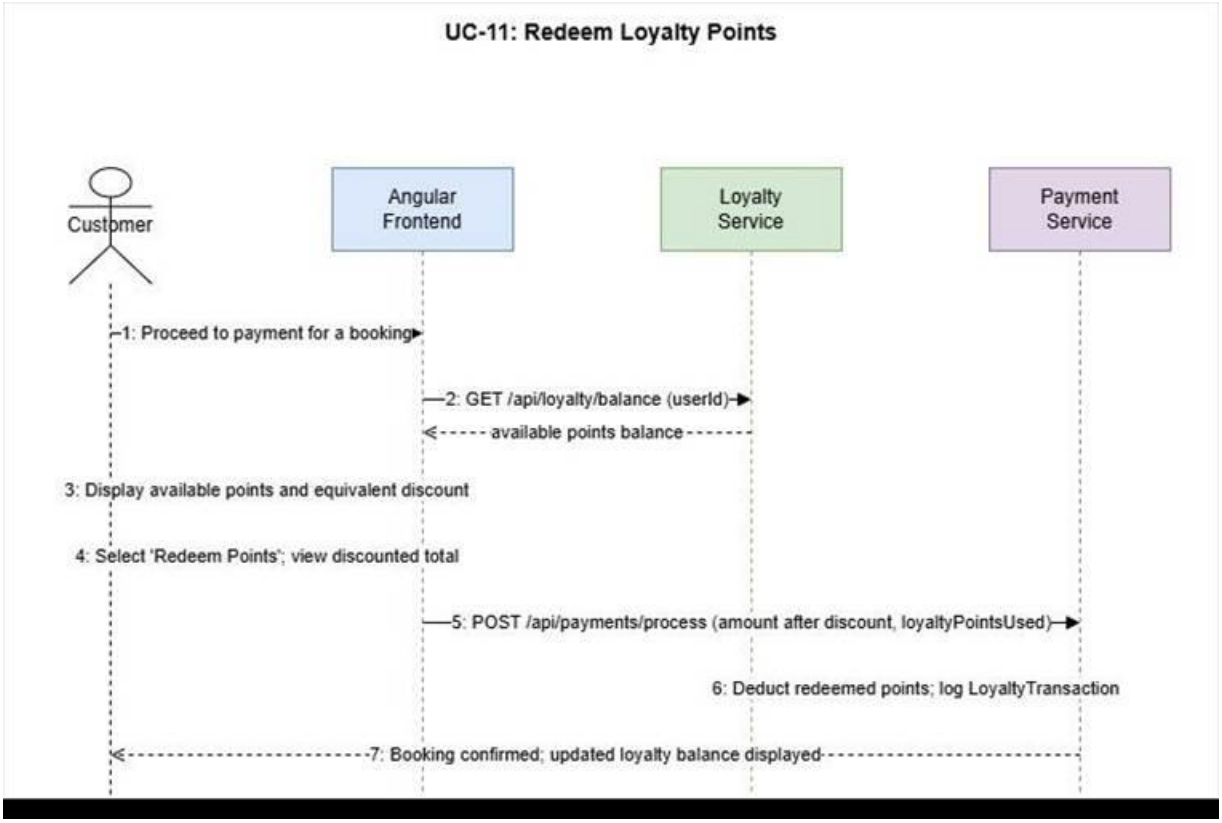


Figure C.10: Sequence – UC-11 Redeem Loyalty Points

C.6 UI/UX Design

C.6.1 Overview

The Gaming Castle web application shall provide a modern, responsive, and accessible user interface. The design follows a dark-themed gaming aesthetic that is consistent with the Gaming Castle brand identity. All pages shall render correctly on desktop, tablet, and mobile devices (minimum viewport width of 320px), in conformance with NFR07.

C.6.2 Design Principles

- Consistency: A unified color scheme, typography (sans-serif headings, readable body text), and iconography shall be applied across all pages.
- Simplicity: Core customer workflows (slot booking, tournament registration, loyalty redemption) shall be completable in a maximum of 5 minutes by a first-time user.
- Feedback: All form submissions, payment processing stages, and booking confirmations shall provide visible status feedback via toast notifications or inline messages.
- Accessibility: The interface shall comply with WCAG 2.1 Level AA, ensuring usability for users with visual or motor impairments.

C.6.3 Key Screens and Wireframe Descriptions

C.6.3.1 Customer – Home Page

The home page shall display a hero section with a call-to-action 'Book a Slot' button, a featured upcoming tournament card, and a loyalty points balance widget for authenticated users. The top navigation bar shall include links to Book a Slot, Tournaments, My Bookings, Loyalty Rewards, and Profile.

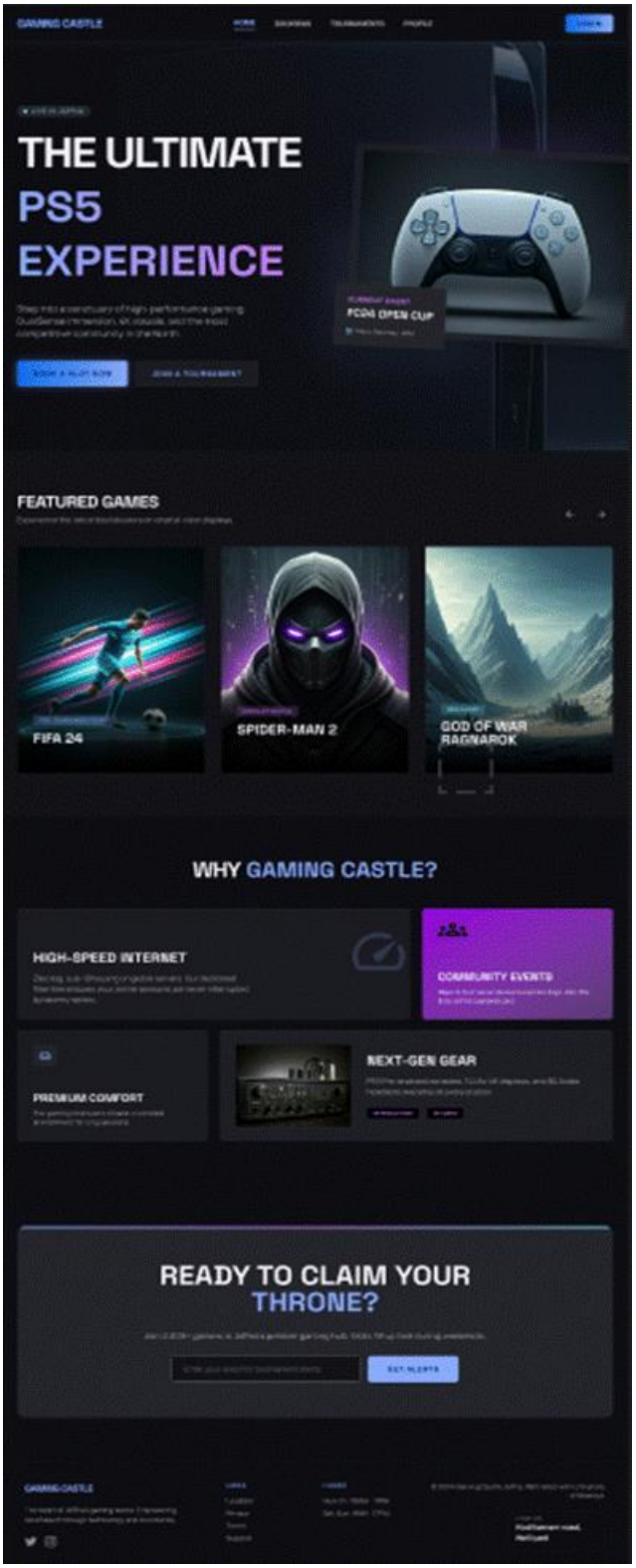


Figure C.11: UI/UX Design - Customer – Home Page

C.6.3.2 Customer – Slot Booking Page

The slot booking page shall present an interactive calendar date picker, time slot grid showing available and occupied slots with color coding (green = available, red = occupied), a console and game selector, a real-time booking summary panel, and a payment and confirm button. Unavailable slots shall be visually disabled and not selectable.

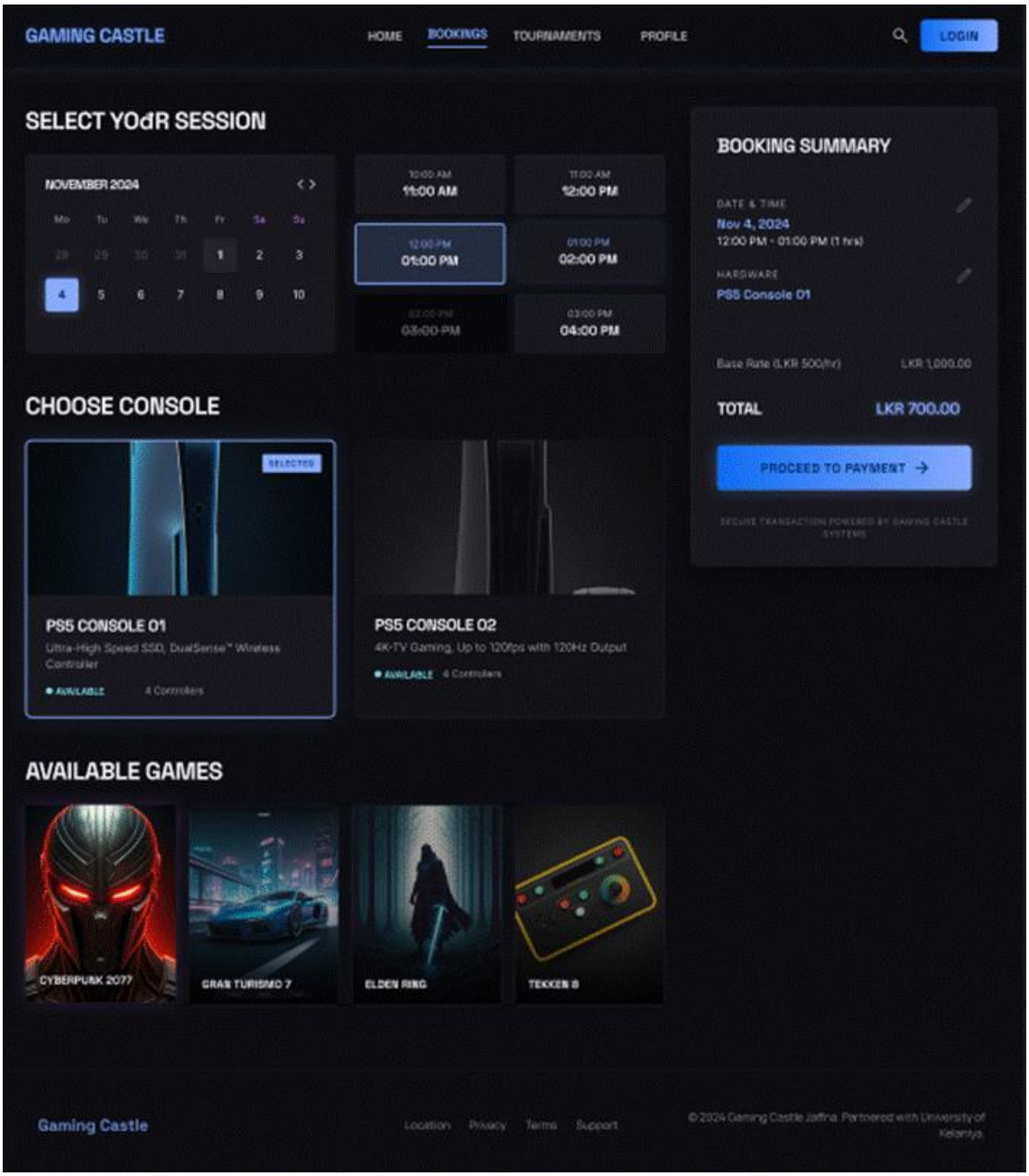


Figure C.12: UI/UX Design - Customer – Slot Booking Page

C.6.3.3 Customer – Payment Page

The payment page shall display the booking summary with the applicable charge, a loyalty points redemption toggle showing the available points balance and resulting discount, a payment method selector (card or gateway), a secure card entry form, and an order total with a confirm payment button.

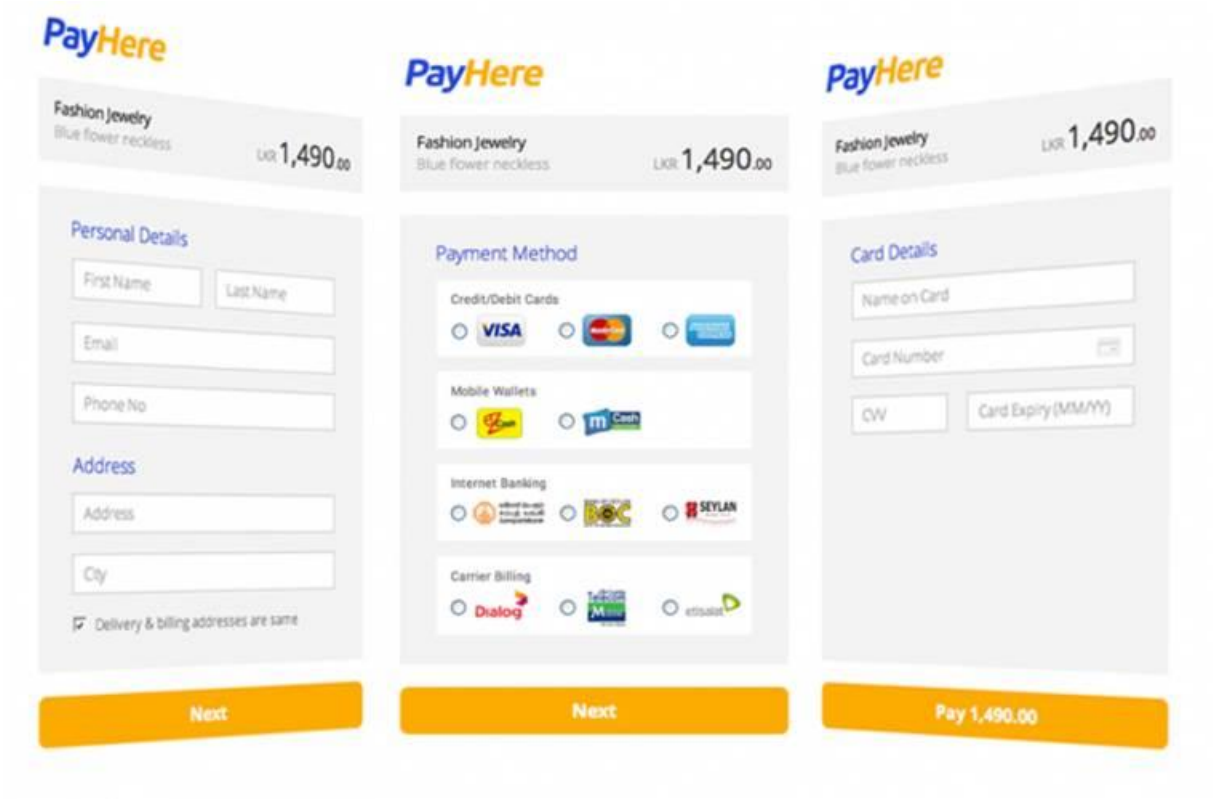


Figure C.13: UI/UX Design - Customer – Payment Page

C.6.3.4 Customer – Tournament Page

The tournaments page shall list all available and upcoming tournaments in card format, showing the game title, date, entry fee, remaining spots, and a register button. Closed or full tournaments shall be displayed with a disabled state and a status badge.

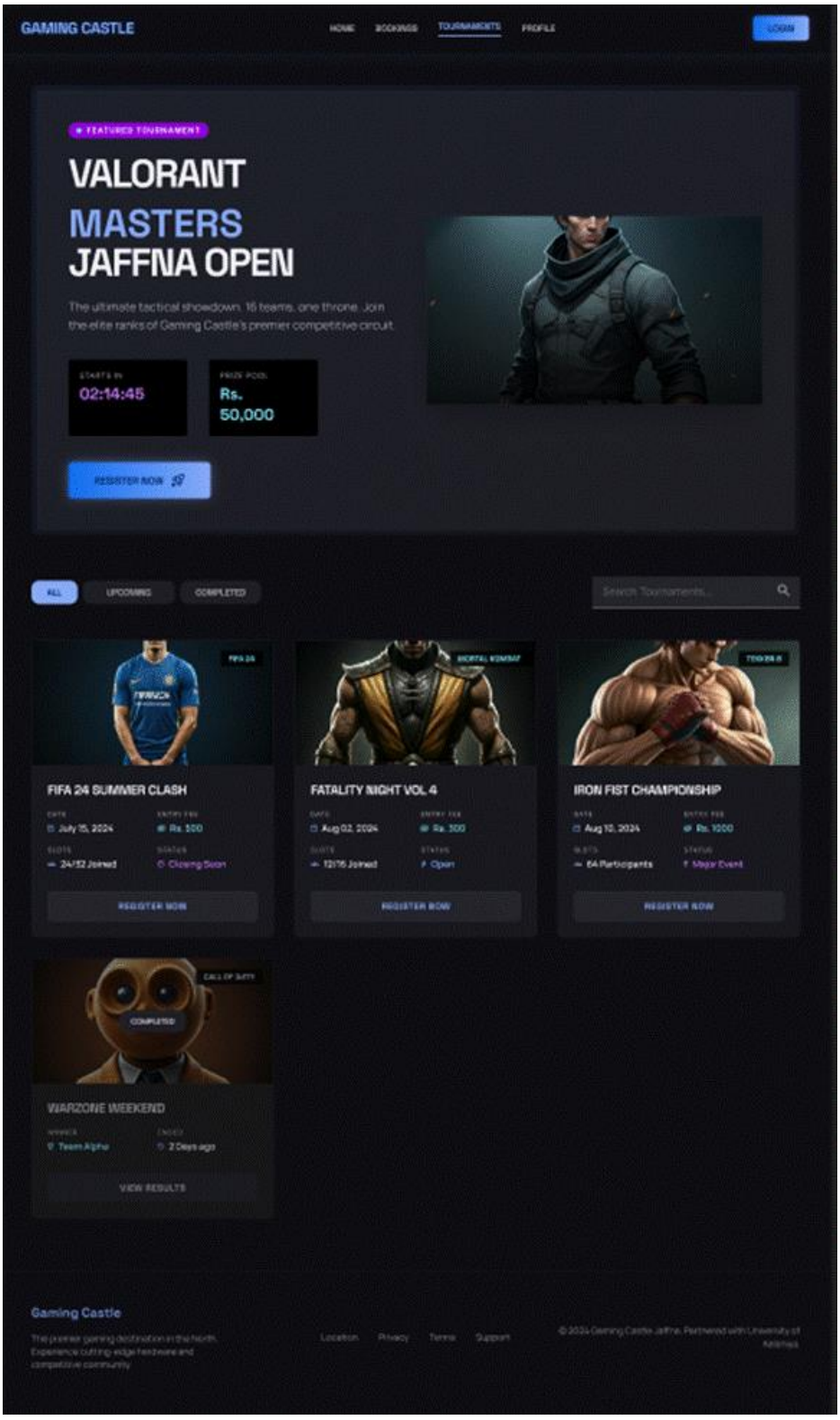


Figure C.14: UI/UX Design - Customer – Tournament Page

C.6.3.5 Admin – Dashboard

The admin dashboard shall provide a summary panel with four key widgets: Today's Bookings (count and revenue), Active Sessions (live console occupancy), Upcoming Tournaments, and Total Registered Users. A left-side navigation panel shall provide access to Bookings, Tournaments, Users, Payments, and Reports sections.

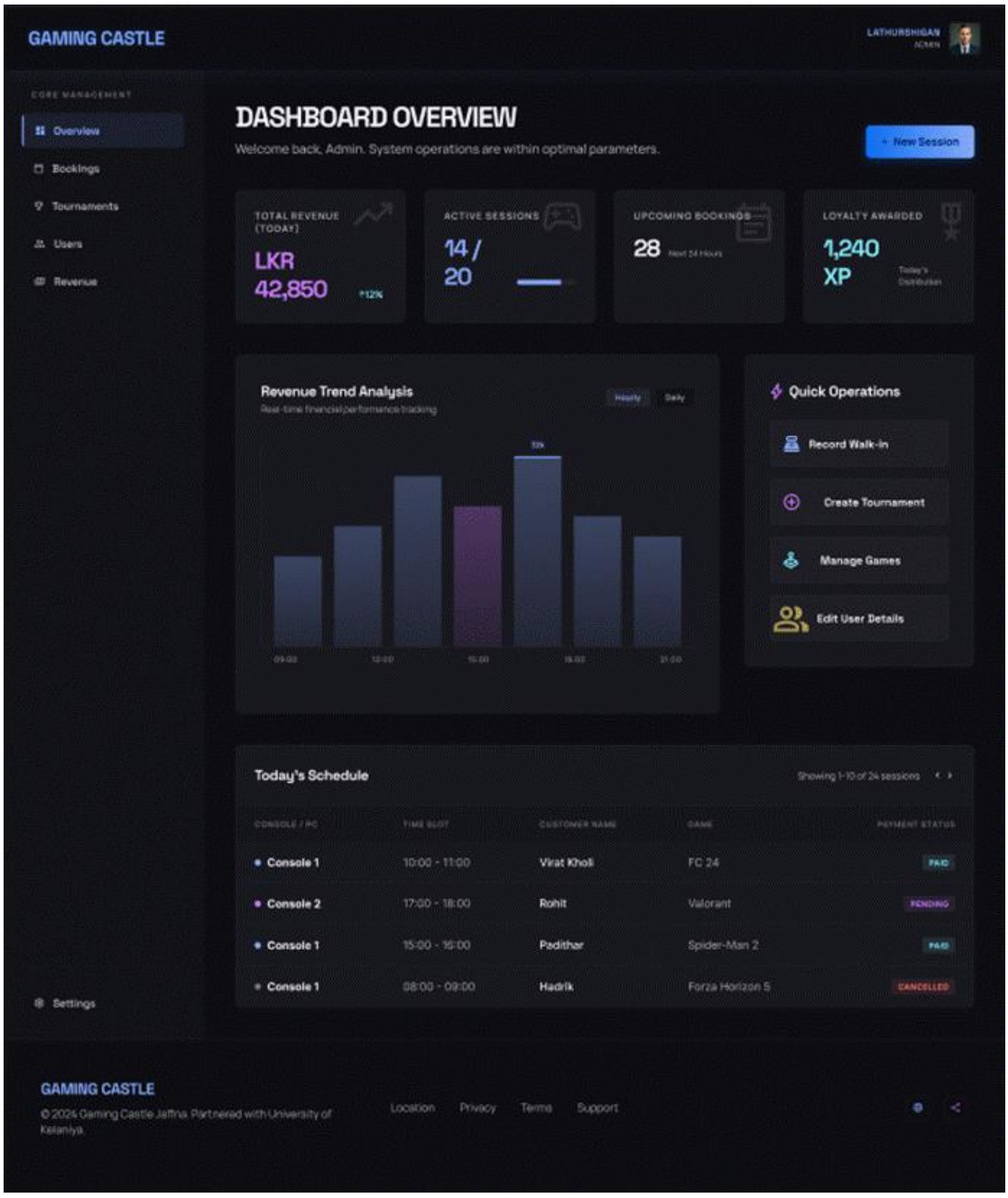


Figure C.15: UI/UX Design - Admin – Dashboard

C.6.3.6 Admin – Booking Management

The admin booking management page shall display a searchable and filterable table of all bookings (date, customer, console, payment status), an inline walk-in booking form, and action buttons for cancel and reschedule. A booking detail panel shall be accessible on row selection.

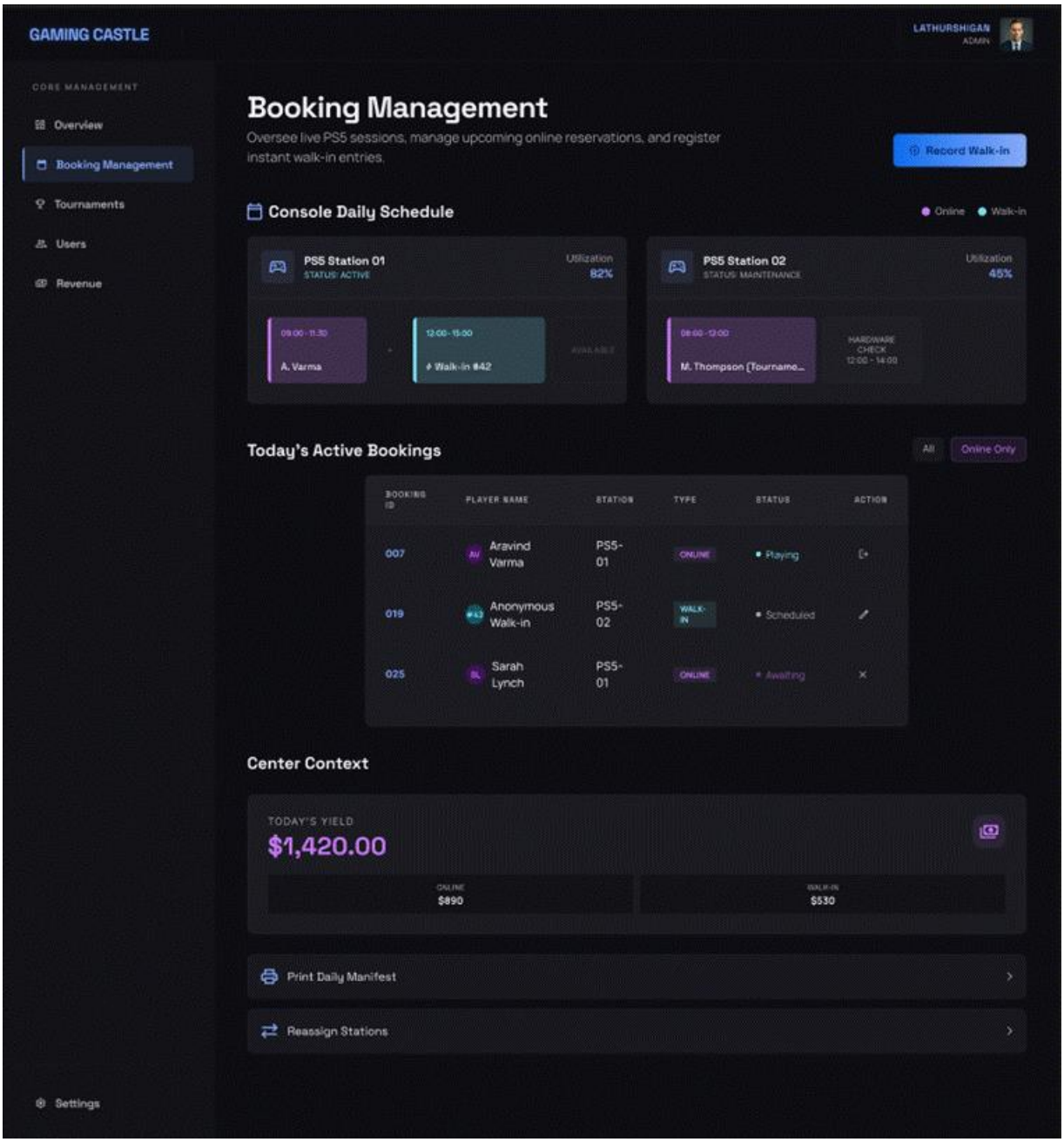


Figure C.16: UI/UX Design - Admin – Booking Management

C.6.4 Navigation Flow

User Role	Navigation Path
Customer	Login → Home → Book Slot → Payment → Confirmation
Customer	Login → Tournaments → Tournament Detail → Register → Payment
Customer	Login → Profile → Loyalty Rewards → Redeem (on Booking page)
Admin	Login → Admin Dashboard → Manage Bookings / Tournaments / Users / Reports
Customer	Login → Logout → Home
Customer	Login → Forgot Password → Enter Email → Reset Link (Email) → Set New Password → Login

Table C.13: Navigation Flow by Role

C.6.5 Accessibility Considerations

The following WCAG 2.1 Level AA conformance requirements shall be implemented:

- All form inputs shall include associated <label> elements and ARIA attributes for screen reader compatibility.

- Color contrast ratios shall meet the minimum of 4.5:1 for normal text and 3:1 for large text.
- All interactive elements (buttons, links, form fields) shall be keyboard-navigable and shall display a visible focus indicator.
- Error messages shall be communicated both visually and via ARIA live regions so that screen readers announce validation failures.
- All images and icons used decoratively shall have an empty alt attribute; informational images shall include descriptive alt text.
- The system shall not rely solely on color to convey information; status indicators shall include text labels (e.g., Available, Occupied).

C.7 Security Design

C.7.1 Authentication Mechanism

Authentication shall be implemented using JSON Web Tokens (JWT). Upon successful login, the User Service shall generate a signed JWT containing the user's ID, role, and expiration timestamp. The token shall be transmitted to the client and stored in a secure HTTP-only cookie to prevent XSS-based token theft.

- The cookie shall be configured with the following attributes:
    - HttpOnly
    - Secure
    - SameSite=Strict
- Token expiry shall be set to 60 minutes for customer sessions and 30 minutes for admin sessions.
- A refresh token mechanism shall be implemented to allow seamless session renewal without requiring the user to log in again. The refresh token shall also be stored in a secure HTTP-only cookie.
- For each request, the JWT shall be automatically included via cookies and validated by the API Gateway before forwarding the request to the appropriate microservice.

C.7.2 Authorization Levels

Role-Based Access Control (RBAC) shall be enforced across all system endpoints. Two roles are defined: CUSTOMER and ADMIN.

Resource / Endpoint	Guest	CUSTOMER	ADMIN
Register / Login	✓	✓	✓
Book Gaming Slot	✗	✓	✗
Admin Walk-in Booking	✗	✗	✓
Register for Tournament	✗	✓	✗
Manage Tournament	✗	✗	✓
View Loyalty Points	✗	✓	✗
View Admin Dashboard	✗	✗	✓
Generate Reports	✗	✗	✓
Manage User Accounts	✗	✗	✓

Table C.14: Role-Based Access Control Matrix

C.7.3 Data Protection Approach

- All customer passwords shall be stored exclusively as bcrypt hashes with a minimum cost factor of 12. Plain-text password storage is strictly prohibited.
- Personally identifiable information (PII) including names, email addresses, and phone numbers shall be stored in compliance with applicable data protection regulations.
- Payment card data shall not be stored within the Gaming Castle system; all card processing shall be delegated entirely to the integrated payment gateway (e.g., Stripe or PayHere), which is PCI DSS compliant.
- Database access credentials shall be stored as environment variables and shall not be hardcoded in source code or committed to version control.

C.7.4 Input Validation and Sanitization Strategy

- All user inputs shall be validated on the client side (Angular reactive forms) and on the server side (Spring Boot Bean Validation) independently. Client-side validation alone shall not be relied upon as a security measure.
- All string inputs shall be sanitized to prevent Cross-Site Scripting (XSS) attacks. Angular's built-in DOM sanitizer shall be used on the frontend.
- All database queries shall use parameterized queries or JPA repositories to prevent SQL injection attacks.
- File uploads, if introduced in future iterations, shall be restricted to whitelisted MIME types and maximum file size limits.
- The system shall undergo an OWASP Top 10 vulnerability assessment prior to production deployment, in conformance with NFR12.

C.7.5 Encryption Standards

Context	Standard	Notes

Data in Transit	TLS 1.2 / 1.3	Enforced for all HTTP communications (HTTPS mandatory)
Password Storage	bcrypt (cost factor 12)	No plain-text or MD5/SHA-1 password hashing permitted
JWT Signing	HMAC-SHA256 (HS256)	Secret key stored in environment variable; RS256 optional for distributed scenarios
Database at Rest	AES-256	Cloud provider managed encryption for the PostgreSQL volume
Session Tokens	HTTP-only Secure Cookies	stored in HTTP-only cookies

Table C.15: Encryption Standards

C.7.6 Account Security

- Brute-force protection: accounts shall be temporarily locked after 5 consecutive failed login attempts, as described in the Login sequence diagram.
- Password reset shall require verification via a time-limited token sent to the registered email address or phone number.
- Admin accounts shall be created exclusively by existing administrators; self-registration for the Admin role shall not be permitted.

C.8 Deployment Design

C.8.1 Hosting Plan

The Gaming Castle system shall be deployed on Amazon Web Services (AWS) using AWS Free Tier for initial academic deployment and scalable paid tiers for production. The following AWS services shall be utilized:

Component	AWS Service	Purpose
Frontend	AWS S3 + CloudFront	Static Angular build served via CDN for low latency
API Gateway	AWS EC2 / ECS Fargate	Container orchestration for Spring Cloud Gateway
Microservices	AWS ECS Fargate	Serverless containers for each microservice
Database	AWS RDS (PostgreSQL)	Managed relational DB with automated backups
Load Balancer	AWS ALB	Application Load Balancer for traffic distribution
Container Registry	AWS ECR	Stores Docker images for each microservice

Table C.16: Hosting Plan

C.8.2 Hardware Requirements

The system is designed for cloud-based deployment and does not require dedicated physical hardware. The recommended minimum specifications per containerized service instance are:

- vCPU: 0.5 vCPU per microservice instance (scalable to 2 vCPU under load)
- Memory: 512 MB RAM per instance (scalable to 2 GB under load)
- Storage: 50 GB SSD for the RDS PostgreSQL instance
- Network: Stable internet connectivity at the Gaming Castle admin terminal (minimum 10 Mbps)

C.8.3 Environment Specification

Environment	Infrastructure	Purpose
Development	Local machine; Docker Compose	Individual feature development and unit testing
Staging	AWS ECS Fargate (reduced capacity)	Integration testing, UAT, and pre-release validation
Production	AWS ECS Fargate (full capacity) + RDS	Live system accessible by Gaming Castle clients

Table C.17: Environment Specification

C.8.4 CI/CD Pipeline Overview

A Continuous Integration / Continuous Deployment (CI/CD) pipeline shall be implemented using GitHub Actions to automate the build, test, and deployment process. The pipeline stages are as follows:

Stage	Tool	Action
1	GitHub Actions	Trigger on push to main or pull request to main branch
2	Maven / npm	Build backend microservices (mvn package) and frontend (ng build)
3	JUnit / Jest	Execute unit tests and integration tests; fail pipeline on test failure
4	Docker	Build Docker image for each modified microservice; tag with commit SHA
5	AWS ECR	Push tagged Docker images to AWS Elastic Container Registry
6	AWS ECS	Deploy updated container images to ECS Fargate (staging auto-deploy; production manual approval)
7	GitHub Actions	Notify team of deployment status via commit status and optional Slack alert

Table C.18: CI/CD Pipeline Stages

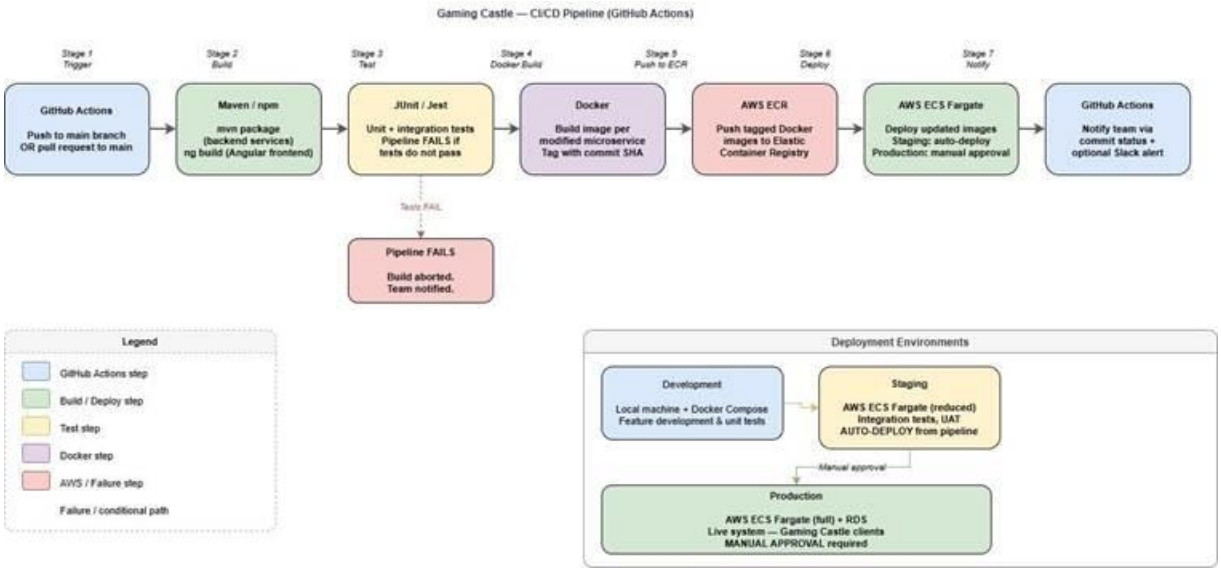


Figure C.17: CI/CD Pipeline Diagram

C.8.5 Network Design

The network architecture shall follow AWS best practices for security and availability:

- The system shall be deployed within an AWS Virtual Private Cloud (VPC) divided into public and private subnets.
- The Angular frontend (S3/CloudFront) and the Application Load Balancer shall reside in the public subnet and be accessible from the internet.
- All microservice containers and the RDS PostgreSQL instance shall reside in private subnets and shall not be directly accessible from the internet.
- Traffic from the internet shall be routed through CloudFront (for the frontend) and the ALB (for API traffic), which forwards requests to the API Gateway container in the private subnet.
- Security Groups shall enforce least-privilege ingress and egress rules: only HTTPS (443) shall be exposed publicly; inter-service communication shall be restricted to the internal VPC CIDR range.

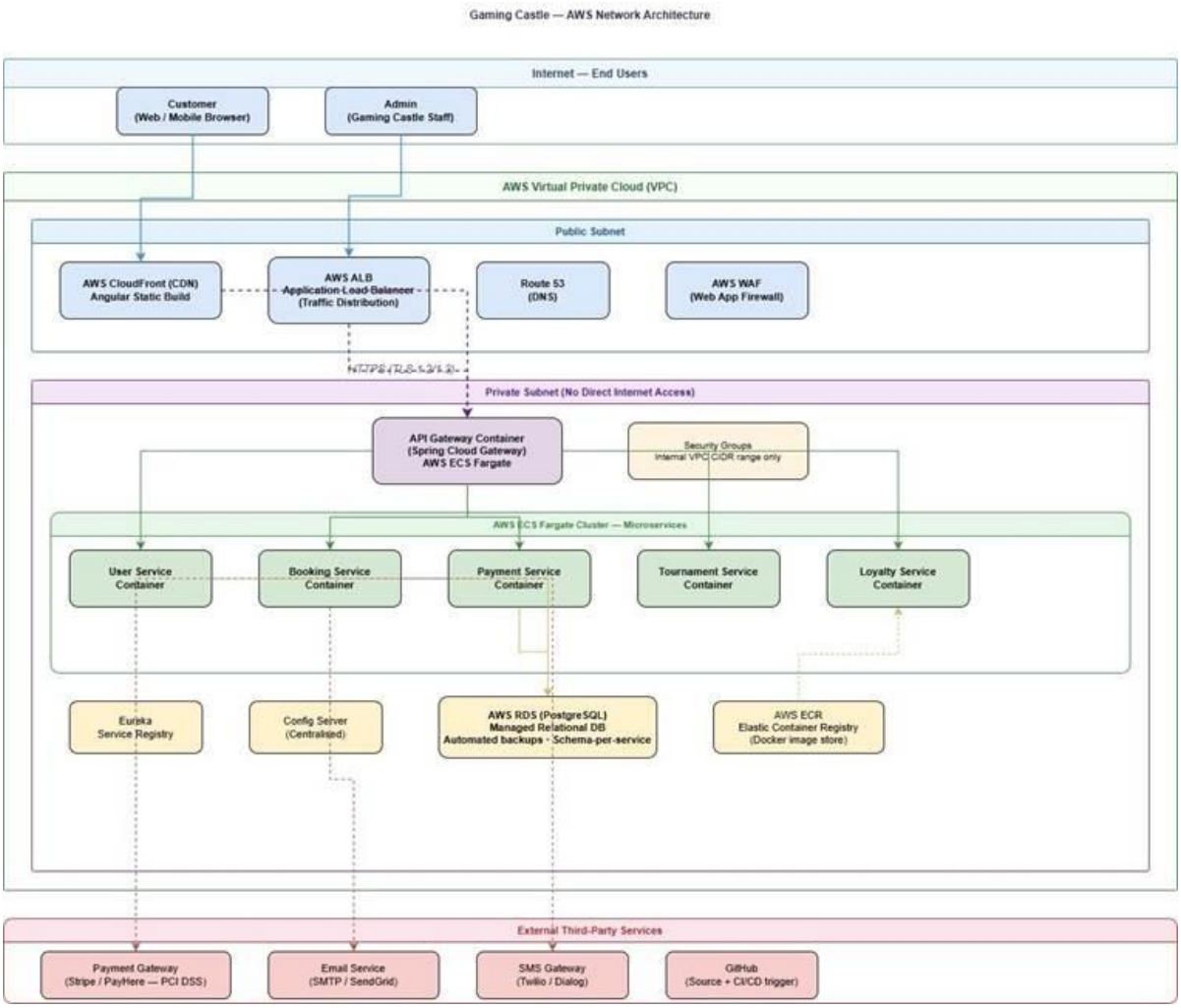


Figure C.18: Network Architecture Diagram

C.9 References

[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[2] ISO/IEC, "Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE)," ISO/IEC 25010:2011, 2011.

[3] OWASP Foundation, "OWASP Top Ten," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. (Accessed Apr. 10, 2026).

[4] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. (Accessed Apr. 10, 2026).

[5] Spring, "Spring Boot Reference Documentation," Pivotal Software, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/. (Accessed Apr. 10, 2026).

[6] Docker Inc., "Docker Documentation," 2024. [Online]. Available: https://docs.docker.com/. (Accessed Apr. 10, 2026).

[7] Amazon Web Services, "AWS Architecture Center," 2024. [Online]. Available: https://aws.amazon.com/architecture/. (Accessed Apr. 10, 2026).

[8] M. Fowler, "Microservices," martinfowler.com, 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html. (Accessed Apr. 10, 2026).

Proofreading Certification

I hereby certify that I have carefully proofread the final draft of this report titled **"Final Design Report"** and have checked for grammatical, spelling, and language accuracy to the best of my knowledge.

To the best of my ability, I confirm that the document is free from major grammatical and spelling errors. However, any remaining errors are the sole responsibility of the student.

Name of Certifier: Rasalingam Ragul

Designation: Software Engineer, Arimac.

Signature: R.Ragul

Date: 30/04/2026